

Project Report

SmartNet: Real-Time Anomaly Detection and Monitoring for Network Traffic

Submitted by

BSCSB-21-06
Shamikh Mushtaq
2021-2025

Supervised by

Dr. M. Muzaffar Hameed



**DEPARTMENT OF COMPUTER SCIENCE
BAHAUDDIN ZAKARIYA UNIVERSITY MULTAN
PAKISTAN**

FINAL APPROVAL

This is to certify that we have read this report titled **SmartNet: Real-Time Anomaly Detection and Monitoring for Network Traffic** submitted by **Shamikh Mushtaq Roll No. BSCSB-21-06** and it is our judgment that this report is of sufficient standard to warrant its acceptance by **Bahauddin Zakariya University, Multan** for the degree of BS (Computer Science).

Committee:

1. External Examiner

<<Examiner Name>>

<<Designation>>

<<Organization>>

2. Supervisor

Dr. M. Muzaffar Hameed

Assistant Professor,

Department of Computer Science

.

3. Chairperson of Department

Dr. Humaira Afzal

Professor,

Department of Computer Science

DEDICATION

To my Loving Parents

ACKNOWLEDGMENT

I would like to express my deepest gratitude to everyone who contributed to the successful completion of my Final Year Project and thesis. First and foremost, I am immensely grateful to my project supervisor, Dr. M. Muzaffar Hameed, and my teachers for their invaluable guidance, insightful feedback, and unwavering encouragement throughout this journey. Their expertise and mentorship were instrumental in shaping this work.

My sincere appreciation goes to my family and friends for their constant support, patience, and motivation during challenging times. Their belief in me kept me focused and determined. I also extend my thanks to my peers and colleagues for their constructive discussions, collaboration, and assistance, which enriched my learning experience.

Lastly, I acknowledge the resources and facilities provided by Bahauddin Zakariya University, Multan, which played a pivotal role in the execution of this project. This achievement would not have been possible without the collective support of all these individuals and institutions.

PROJECT BRIEF

PROJECT NAME	SmartNet: Real-Time Anomaly Detection and Monitoring for Network Traffic
ORGANIZATION NAME	Bahauddin Zakariya University, Multan
UNDERTAKEN BY	Shamikh Mushtaq
SUPERVISED BY	Dr. M. Muzaffar Hameed
STARTING DATE	November 15, 2024
COMPLETION DATE	April 26, 2025
COMPUTER USED	Intel Core i7 7th Gen, 2.7 GHz, 16GB RAM, 500GB HDD + 256GB SSD
OPERATING SYSTEM	MS Windows 11 Pro
SOURCE LANGUAGE(S)	Python, Html, CSS, Javascript
DBMS USED	My-SQL
TOOLS/PACKAGES	Flask, Visual Studio, Pandas, Numpy, XGBOOST, Scikit Learn, Chart.js, Matplotlib, Seaborn, tkinter

ABSTRACT

This project presents the design and implementation of SmartNet, an advanced real-time network traffic analyzer and threat detection system. The goal of the system is to enhance cybersecurity by monitoring network activity and identifying potential threats, including malicious and suspicious traffic. SmartNet captures and analyzes network traffic, specifically TCP and UDP packets, and extracts relevant features from each flow. It utilizes a machine learning model, XGBoost, to classify network flows as normal, suspicious, or malicious. A web-based admin panel facilitates live threat monitoring, manual IP blocking, access to flow history, and analysis of system resources. Additionally, the system includes a Tkinter-based GUI that streamlines the initiation of background services. The platform offers real-time insights such as packet rates, TCP/UDP distribution, and detected threats. It also provides support for filtering, searching, and exporting flow data for further analysis. This integrated solution combines machine learning, network monitoring, and intuitive visualization to support proactive network defense in a single, user-friendly interface.

Table of Contents

Contents	Page No.
Chapter 1	
Introduction	
1.1 Project Introduction	1
1.1.1 Main Theme	1
1.1.2 Scope of the Project	1
1.1.3 Objectives of the Project	2
1.2 Introduction to Organization	3
1.2.1 Organizational Setup and Structure	3
1.2.2 Main Aim and Work Environment	3
1.3 Conclusion and Future Prospects	4
Chapter 2	
System Analysis	
2.1 Feasibility Study	5
2.2 Existing System: Data Gathering	5
2.2.1 Questionnaires	5
2.2.2 Sampling and Observations	6
2.3 Existing System: Data Analysis	6
2.4 Data Flow Diagrams	8
2.4.1 Level 0 DFD	8
2.4.2 Level 1 DFD	8
2.4.3 Level 2 DFD	9
2.5 Requirements Engineering	10
2.6 Deliverables	11
Chapter 3	
System Design	
3.1 Introduction to System Design	12
3.2 Proposed System and its Features	12
3.3 System Design using UML	13
3.3.1 Use Case Diagram	14

3.3.2 Packet Capture and Prediction Sequence Diagram	16
3.3.3 Admin Login and Authentication Sequence Diagram	18
3.3.4 Real-Time Alert Notification Sequence Diagram	19
3.3.5 Admin Viewing Traffic Stats and Logs Sequence Diagram	21
3.3.6 Resource Monitoring Panel Sequence Diagram	22
3.3.7 IP Blocking/Unblocking Sequence Diagram	23
3.4 Database Design	25
3.5 Integration and Data Flow	27
3.6 Entity Relationship Diagram (ERDs)	28
3.7 Building a Relational Model from ERDs	32

Chapter 4

System Development & Implementation

4.1 Introduction to System Development	35
4.2 Tools/Languages Selection	35
4.3 Hardware for the System	36
4.4 Software Development & Implementation	37
4.4.1 Client-Side Technology	37
4.4.2 Server-Side Technology	38
4.5 Code/Algorithms of Important Modules	38
4.5.1 XGBOOST Model for Packets Prediction	38
4.5.2 Main Module for Capturing Packets and Handling Tasks	41
4.5.3 Flask Server Function for Making Predictions	44
4.5.4 Flask Server for Saving Packets to Database	48

Chapter 5

User's Guide

5.1 Input Forms	53
5.1.1 Admin Authentication Form	53
5.1.2 Password Recovery Form	54
5.1.3 Update Password Form	56
5.1.4 Update Profile Picture Form	57
5.1.5 System Services Management GUI	58

5.2 Reports	60
5.2.1 TCP vs UDP Packets Graph	60
5.2.2 Packets per Second Line Graph	61
5.2.3 Recent Threats Table	62
5.2.4 ML Model Details Page	63
5.2.5 System Resources Monitoring Page	67
5.2.6 Complete Flow Data Analysis	68
5.2.7 IP Restrictions Panel	69

Chapter 6

Conclusion

6.1 Summary of the Project	71
6.2 Key Contributions	71
6.3 Limitations	72
6.4 Future Work	73
6.5 Final Remarks	74

References	75
-------------------	----

Chapter 1

Introduction

1.1 Project Introduction

SmartNet is a real-time anomaly detection system which is focused on enhancing security by monitoring network traffic using Machine Learning algorithms to detect malicious traffic by distinguishing it from the normal one. It helps Network Administrators quickly determine potential network attack/malicious traffic. It works by analyzing the incoming traffic in real-time and learning how normal traffic looks like. By doing so, it sets the boundary for normal traffic and malicious traffic. If the borderline is crossed by incoming traffic, then it means something's wrong, i.e. traffic is malicious. It then helps the admin/organization to protect the network by taking necessary measures.

1.1.1 Main Theme

SmartNet operates as an advanced software solution which monitors the real-time traffic. It uses machine learning algorithms to evaluate the incoming traffic on the network to detect the abnormal traffic. Primarily, SmartNet monitors deviations from the normal traffic, if traffic deviates from normal then it is marked as malicious and an alert is sent to the admin and also stored in logs for later analysis.

Machine Learning is what makes it unique from other software solutions. Using previously learned data, it alerts the admin if the traffic is malicious otherwise it remains silent, i.e., working in the background. It has a simple interface on which admin can login, delete logs and view logs related to current and previous malicious traffic.

1.1.2 Scope of the Project

The main objective of the SmartNet is to respond to monitor the increased network activity that will help to detect the malicious incoming traffic. The scope of the project is as follows:

- **Real-time Traffic Monitoring:** It is the monitoring and analysis of network traffic relative to predefined normal and malicious traffic behaviors.
- **Machine Learning-based Anomaly Detection:** SmartNet will use the predefined learning techniques to always monitor the network data to build the baseline of normal traffic. If traffic deviates from this baseline then it's marked as malicious.

- **Anomaly Detection:** If an anomaly is detected then the network administrator is notified of it immediately for quick action.
- **Log Management and Analysis:** The identified anomaly will be logged into the MySQL database for later analysis which would be used for investigation and identifying possible patterns purposes.
- **Admin Interface:** A user-friendly interface would be designed for the admin to manage SmartNet and to view real-time traffic and logs stored in the database.

This system is designed to work as a server-side application which would be integrated into the existing network infrastructure and scaled according to the particular organization needs.

1.1.3 Objectives of the Project

Following are the key objectives of the SmartNet:

- **Design a System for Continuous Monitoring of Network Traffic:** Develop a system that will continuously monitor and analyze the real-time traffic.
- **Machine Learning Integration:** Implement Machine Learning algorithms to learn malicious network behavior to ensure precise anomaly detection.
- **Alert and Notification Framework:** Effective design to alert the admin about the security violations at the moment they are discovered.
- **Data Logging:** Set a mechanism to collect and store the observed anomalies in MySQL Database for further study.
- **Web Interface Development:** Develop an easy-to-use web interface that lets the admin manage the SmartNet Configurations and access logs.
- **Scalability and Flexibility:** The system would be scalable enough to handle network traffic from any size of networks and also customizable according to the needs of organizations.
- **Security:** Ensure that the system is completely safe. The traffic that is transmitted within the system is encrypted. The Web interface is secure to unauthorized access.

1.2 Introduction to Organization

Bahauddin Zakariya University is a public institution located in Multan, South Punjab, Pakistan. It was established in 1975 and was named after a Sufi saint Bahauddin Zakariya. BZU stood out as one of the best higher educational institutions in South Punjab. It has the largest IT network among all of the South Punjab institutions. It is providing all levels of study whether bachelors, masters or PhDs. BZU Computer Science Department offers a good basis for learning Computer Science both theoretically and practically. Although it does not have extensive research facilities, it equips the students with all the resources to follow their academic choices.

1.2.1 Organizational Setup and Structure

BZU Computer Science Department has experienced faculty guiding the students on technical subjects. My SmartNet project is being developed under the supervision of Dr. M. Muzaffar Hameed who guided and mentored me on each step of it. This kind of arrangement fosters the self-driven learning. The student does the complete work while getting the guidance from the Supervisor when necessary.

1.2.2 Main Aim and Work Environment

The main aim of BZU Computer Science Department is that it not only provides students with the theoretical knowledge but also the relevant practical skills to solve the real-world problems.

As part of my SmartNet project, I work alone under the supervision of Dr. M. Muzaffar Hameed. The working environment is basic but it provides ample facilities such as Computer Labs, required software and guidance. Although the department does not have extensive research facilities, it provides an encouraging environment that helps the students to successfully complete the academic work and projects.

1.3 Conclusion and Future Prospects

SmartNet is an important milestone in this era where security-related issues arise each second around the globe. This ML-based system would give alerts to the admins if malicious traffic is encountered. This would help the admins to respond to it quickly. It would also store such traffic in the database so that it can be analyzed later for investigation and study to enhance the security by learning the flaws.

Future Prospects

In the future, it can be modified for possible expansion and improvement.

- **Advanced Machine Learning Frameworks:** In future, detection model would be improved by replacing the ML algorithms with more accurate algorithms such as deep learning. This would help manage the increased network traffic issues.
- **Integration:** Currently, it's a standalone software solution but later it would be modified to integrate in the firewalls and Intrusion Detection Systems which would enhance its purpose even more.
- **Cloud-based Environment:** Moving the system to a Cloud-based environment will offer scalability and flexibility especially to larger organizations with distributed network infrastructure.
- **Self-Improving System:** Through the development of ML algorithms, SmartNet may self-improve its anomaly detection framework. This would enable it to continuously adapt to dynamic network environment.

In conclusion, SmartNet is a foundational system which can be enhanced to meet the increasing organization requirements. As it's an ML-based system, it would be able to be modified with the development of advanced algorithms.

Chapter 2

Systems Analysis

2.1 Feasibility Study

The feasibility study is the part of the project which determines whether the proposed SmartNet system is feasible and achievable within given constraints. It includes all technical, operational, and financial aspects of the project to make SmartNet.

This study can be divided into three sections:

- 1- **Technical Study:** As the required modern machine learning algorithms and technologies do exist. All the required tools and technologies such as Python and its libraries like Pandas, Scikit-learn, and network traffic analysis utilities are available with sufficient support. The required hardware support is also available, so the project is viable technically.
- 2- **Operational Feasibility:** SmartNet is also operationally feasible as it doesn't disrupt existing infrastructure. Continuous traffic monitoring using machine learning doesn't disrupt the normal traffic flow. The alert mechanism along with logging functionality ensures the admin can respond to malicious traffic quickly with minimal additional burden on the system.
- 3- **Financial Feasibility:** The financial feasibility of SmartNet is also positive as it doesn't require expensive infrastructure. The required tools and technologies are freely available. Additionally, the system can be deployed on existing servers without the need for any special hardware.

2.2 Existing System: Data Gathering

Data gathering is a crucial part of system designing. Extensive data gathering was done to understand the network traffic and anomalies that must be identified. On-site organizational visits were not performed. The following sources and methods were used for data collection:

2.2.1 Questionnaires

As direct observations were not taken by on-site organizational visits, questionnaires were distributed to network administrators to understand pain areas with the existing network monitoring systems. The questionnaires were based on areas such as false positives, less

effective threat detection, and late alerting. Based on the feedback gathered, areas such as distinguishing normal from malicious traffic were critical for making the SmartNet system.

2.2.2 Sampling & Observations

For data sampling, the CIC-IDS 2017 dataset, which is publicly available, was used. It contains both benign and malicious traffic data. It includes traffic data of different attacks such as DoS, DDoS, and other malicious behaviors, as well as normal traffic data. This data was used for training and evaluating the detection model. This dataset served as a representative sample, allowing the system to learn how to differentiate between normal and malicious traffic accurately.

2.3 Existing System: Data Analysis

Many systems and methodologies already exist within network traffic analysis and anomaly detection tools. The developed systems focus on the inspection of network traffic, identifying malicious traffic, and generating alerts on potential security threats. Several established systems and methodologies are often used for data analysis in network traffic monitoring, such as:

1. Statistical-Based Methods

This method is based on analyzing traffic data through statistical techniques to create a normal network behavior baseline. Any traffic differing from the baseline is then identified and marked as malicious. Statistical methods such as means, variances, and standard deviations are used to monitor packet flow. However, such simple methods are not suitable for detecting complex and emerging attack patterns.

2. Machine Learning-Based Methods

Due to the ability of machine learning techniques to learn from data and adapt to emerging attacks, they have gained much attention in the past few decades. Systems implementing K-means clustering, SVM, and neural network-based methods are widely adopted. These systems are trained with historical datasets to distinguish between benign and malicious traffic. For example, the CIC-IDS 2017 dataset is used within many research studies for developing machine learning models. While machine learning-based methods are highly accurate, they require large datasets and significant computing power for training.

3. Signature-Based Detection

Signature-based anomaly detection works by comparing network traffic with a predefined set of attack patterns. These systems are effective in detecting known threats but cannot detect zero-day attacks. Despite their limitations in dynamic environments, they are still widely used in traditional firewalls and IDSs.

4. Hybrid Approaches

Many modern systems combine statistical and machine learning techniques. For example, a hybrid system may use rule-based filtering for preliminary traffic screening and then apply machine learning models for further analysis. This improves detection accuracy while reducing false positives.

5. Real-Time Anomaly Detection Systems

Real-time detection is a requirement to reduce and diminish the effects of attacks. Immediate actions must be taken when an anomaly is detected. Some systems process traffic in real-time and report anomalies immediately. However, detecting anomalies accurately while avoiding false positives remains a major challenge.

Although existing solutions provide a robust foundation for network security, there is still a significant opportunity to improve the precision, scalability, and flexibility of anomaly detection mechanisms. SmartNet would incorporate elements from existing solutions to optimize and enhance anomaly detection systems. The CIC-IDS 2017 dataset contains normal and malicious traffic data, which would be used to train the model to identify both known and unknown threats.

2.4 Data Flow Diagrams (DFDs)

Data Flow Diagrams are used to illustrate the flow of data within the system, helping to visualize how information is processed and transferred between different components. The DFDs for the SmartNet are as follows:

2.4.1 Level 0 DFD

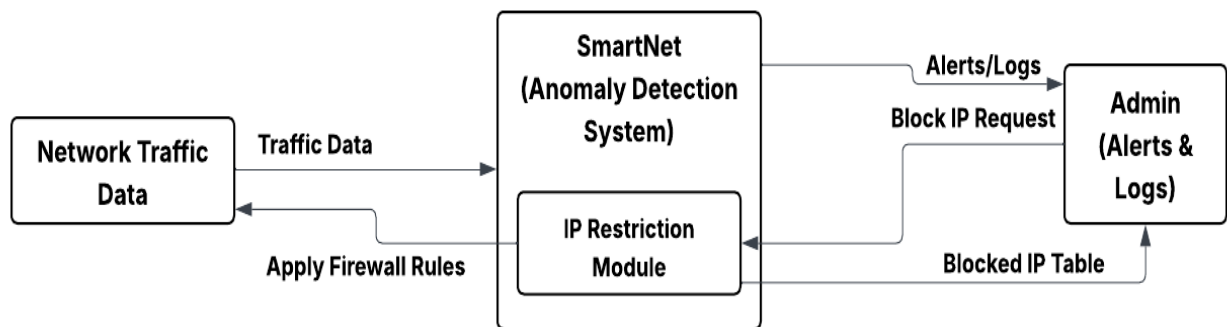


Fig 2.4.1 Level 0 DFD

2.4.2 Level 1 DFD

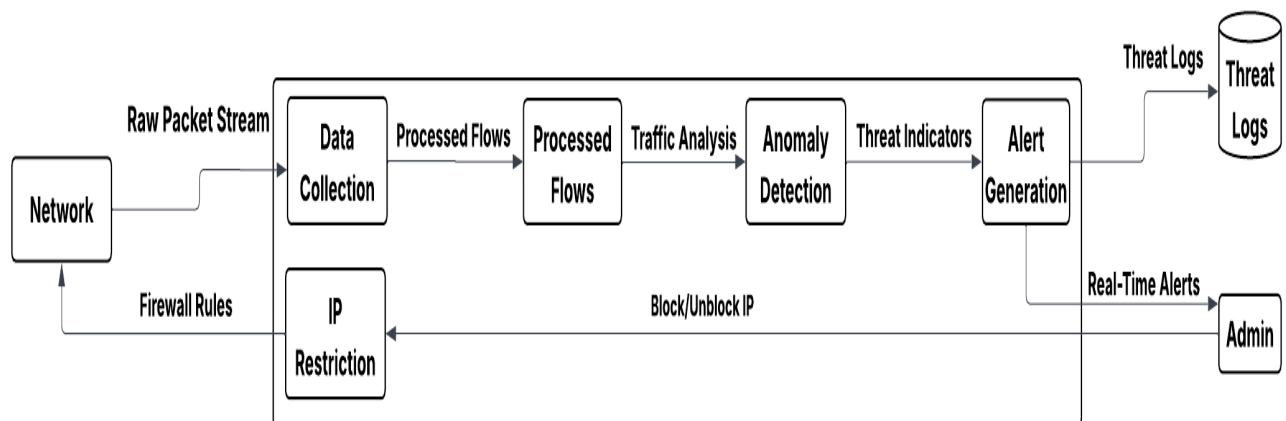


Fig 2.4.2 Network Traffic Anomaly Detection Process - Level 1 DFD

2.4.3 Level 2 DFDs

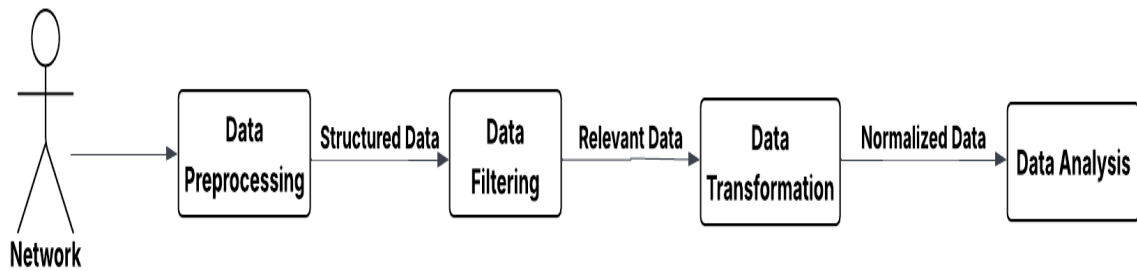


Fig 2.4.3 (a) Network Traffic Data Processing Pipeline – Level 2 DFD

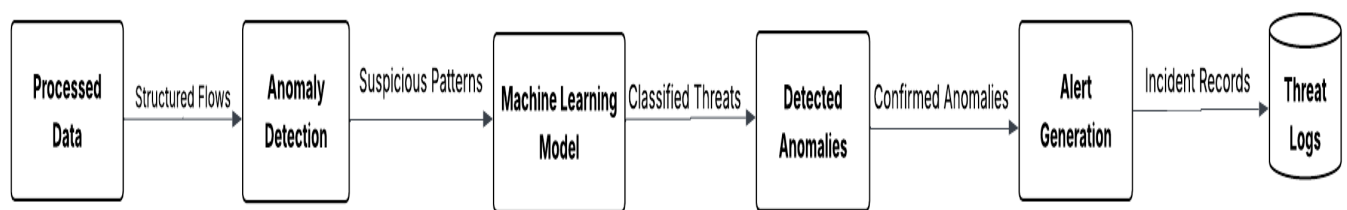


Fig 2.4.3 (b) Anomaly Detection & Classification Pipeline - Level 2 DFD

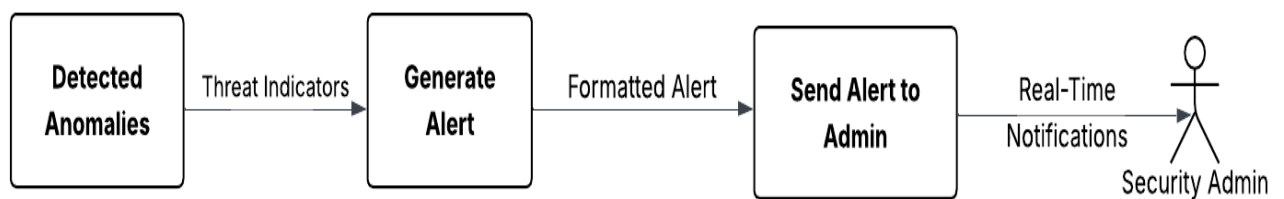


Fig 2.4..3 (c) Real-Time Threat Alert System - Level 2 DFD

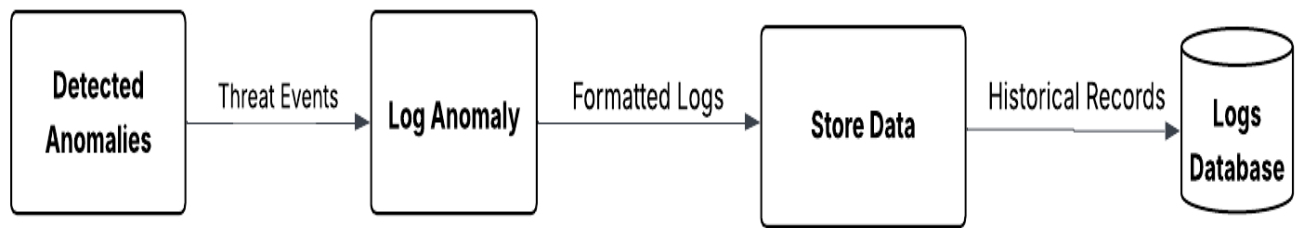


Fig 2.4.3 (d) Anomaly Logging Subsystem - Level 2 DFD

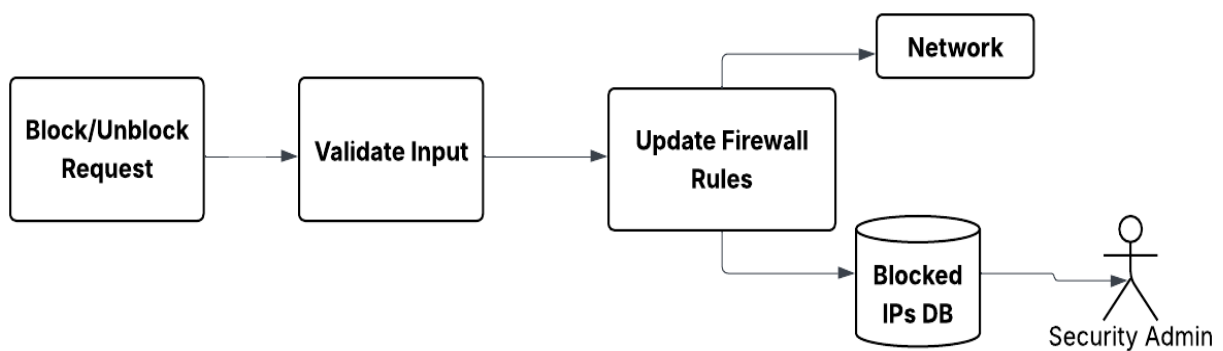


Fig 2.4.3 (e) IP Restriction Module - Level 2 DFD

2.5 Requirements Engineering

Requirements engineering is the process of gathering and defining the functional and non-functional requirements for SmartNet.

Functional Requirements

These define what the system should do, such as:

- Real-time monitoring of network traffic
- Detecting anomalies using a machine-learning-trained model
- Sending alert messages to admins when an anomaly is detected
- Storing logs for historical analysis and future diagnosis

Non-Functional Requirements

These include performance aspects of the system, such as:

- **High Accuracy:** Ensuring precise anomaly detection
- **Low Latency:** Processing network traffic with minimal delay
- **Scalability:** The ability to handle very large volumes of network traffic

The collection of requirements included a review of previously deployed anomaly detection systems in networks, questionnaires distributed to various IT professionals, and an analysis of the CIC-IDS 2017 dataset to identify common types of network anomalies.

2.6 Deliverables

The principal outputs of the SmartNet project are as follows:

1. **Working Prototype:** A system that monitors network traffic in real-time, detects anomalies via a machine learning approach, and sends alerts to administrators.
2. **Data Pipeline:** An integrated pipeline for network flow data acquisition, processing, and analysis.
3. **Documentation:** Technical explanations on setting up and operating the system in its entirety, including how to configure network monitoring and handle alerts.
4. **Final Report:** A comprehensive document outlining the design, architecture, methodology, results, and potential improvements of the system.

These deliverables will provide an integrated solution for real-time anomaly detection in network traffic, offering both practical applications and theoretical insights.

Chapter 3

System Design

3.1 Introduction to System Design

The system design phase is a vital component in the creation of SmartNet: Real-Time Anomaly Detection and Monitoring for Network Traffic, as it lays the groundwork for translating theoretical ideas into an operational system. The process includes the definition of the overall architecture, identification of the required components, and definition of interactions between these components to ensure smooth functioning.

In the context of SmartNet, the emphasis during the system design is creating an efficient and scalable architecture that can process network traffic, identify anomalies through the use of machine learning models, and provide alerts to system administrators. Major considerations during this process involve ensuring the ability to process data in real-time, associating the system with logging databases, and creating an administrator-friendly interface.

In addition, the system design phase not only creates a tangible plan of action but also ensures that the system meets performance, reliability, and security requirements, in line with the goals of the project.

3.2 Proposed System and its Features

The suggested system, SmartNet: Real-Time Anomaly Detection and Monitoring for Network Traffic, is a network security solution designed to monitor, detect, and alert administrators of anomalous network traffic in real time. The system utilizes machine learning methods to analyze traffic patterns, which allows for differentiation between normal and malicious activity. By integrating alert mechanisms with a logging database, the system creates a comprehensive platform for real-time monitoring and post-event analysis.

The core objective of SmartNet is to enhance network security through proactive detection of potential threats, thus reducing response times and enabling administrators to take prompt action. Its robust and scalable design makes it suitable for varied organizational environments, thus ensuring efficiency and reliability.

Features of the Proposed System

1. Real-Time Monitoring

It constantly monitors network traffic for anomalies, thus ensuring instant threat detection.

2. Machine Learning-Based Anomaly Detection

It utilizes machine learning algorithms to develop a baseline of normal network behavior, which allows for the detection of deviations that indicate malicious activity.

3. Alert Generation

The system sends immediate notifications to administrators through an easy-to-use interface whenever an anomaly is detected.

4. Logs Database

It maintains a complete record of all detected anomalies for future analysis and investigation.

5. User-Friendly Admin Interface

The system provides an easy-to-use interface for analyzing real-time alerts and historical information, thus ensuring effective threat management.

6. Scalable and Reliable Architecture

It is designed to support varying volumes of network traffic, thus ensuring adaptability to unique organizational needs.

The suggested system not only overcomes the constraints of existing solutions but also offers an effective, automated approach to safeguarding network infrastructure against potential threats.

3.3 System Design using UML

Unified Modeling Language offers a standardized way to visualize the design and structure of object-oriented systems. For the development of the SmartNet Real-Time Network Traffic Analyzer, UML diagrams were used to represent the interaction between system components and the flow of data within the system. This section includes two key UML diagrams: a Use Case Diagram to illustrate the system's functional requirements from the admin's perspective, and a Sequence Diagram to depict the sequence of operations involved in capturing, analyzing, and storing network traffic.

3.3.1 Use Case Diagram

Actors

- **Admin:** Interacts with the web panel, monitors traffic, manages profile and settings.
- **System:** Includes the Python packet capturing script and Flask backend that automates traffic analysis, prediction, and alerting.

Use Cases

For System

- Capture TCP/UDP Packets
- Extract Features & Convert to Flow
- Predict Packet Type using XGBoost
- Store Packets in MySQL Database
- Send Alert to Admin for Suspicious/Malicious Traffic
- Repeat Process Every 60 Seconds

For Admin

- Login
- View Profile / Change Password
- View Live Threats (Suspicious & Malicious Packets)
- View Traffic Stats (TCP, UDP, Top IPs)
- View Traffic Rate
- View Packet History (Full DB Logs)
- View ML Model Info (Characteristics, Accuracy)
- View Resource Usage (CPU, RAM, Requests/sec)
- Block/Unblock IP

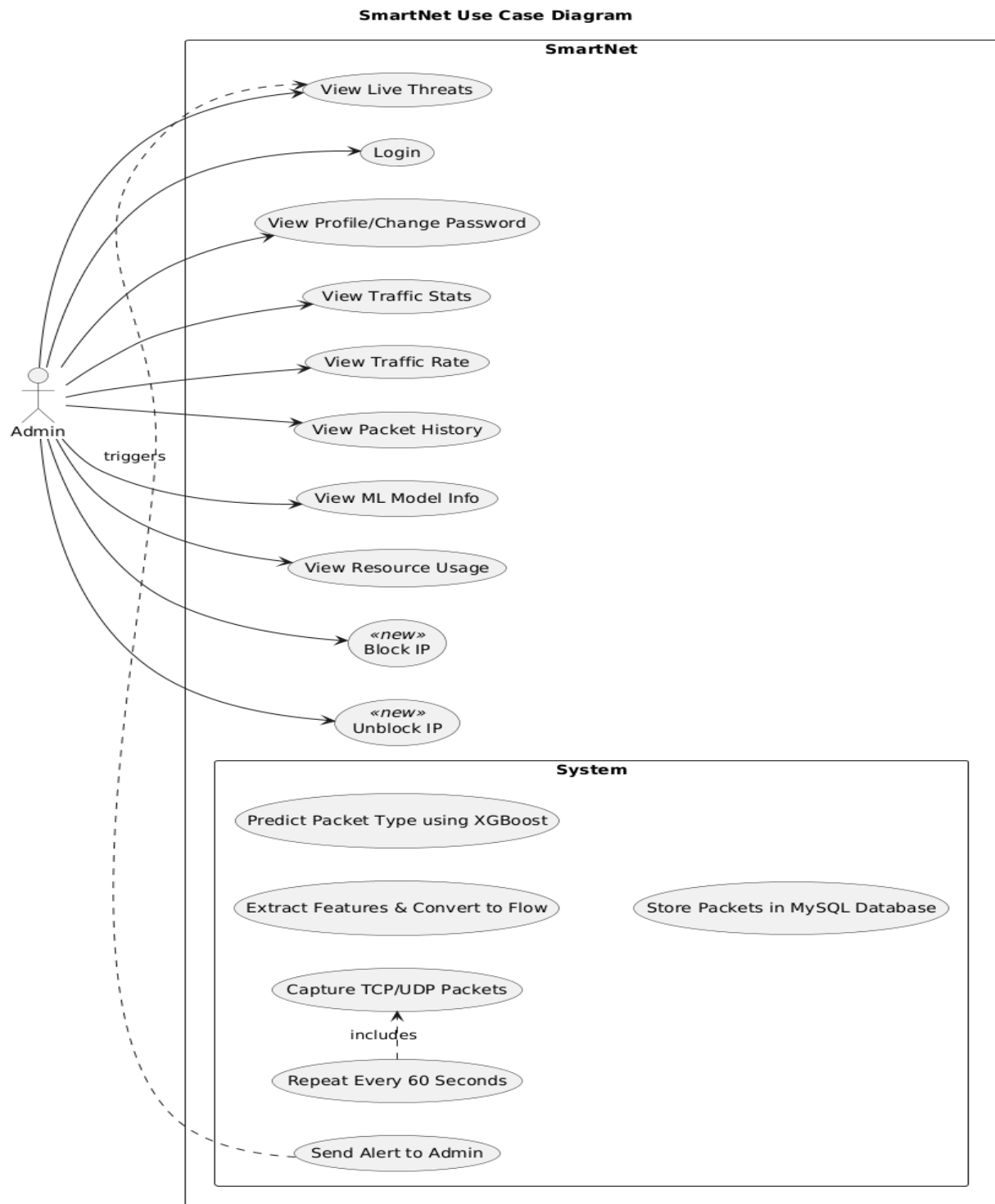


Fig 3.3.1 SmartNet Use Case Diagram

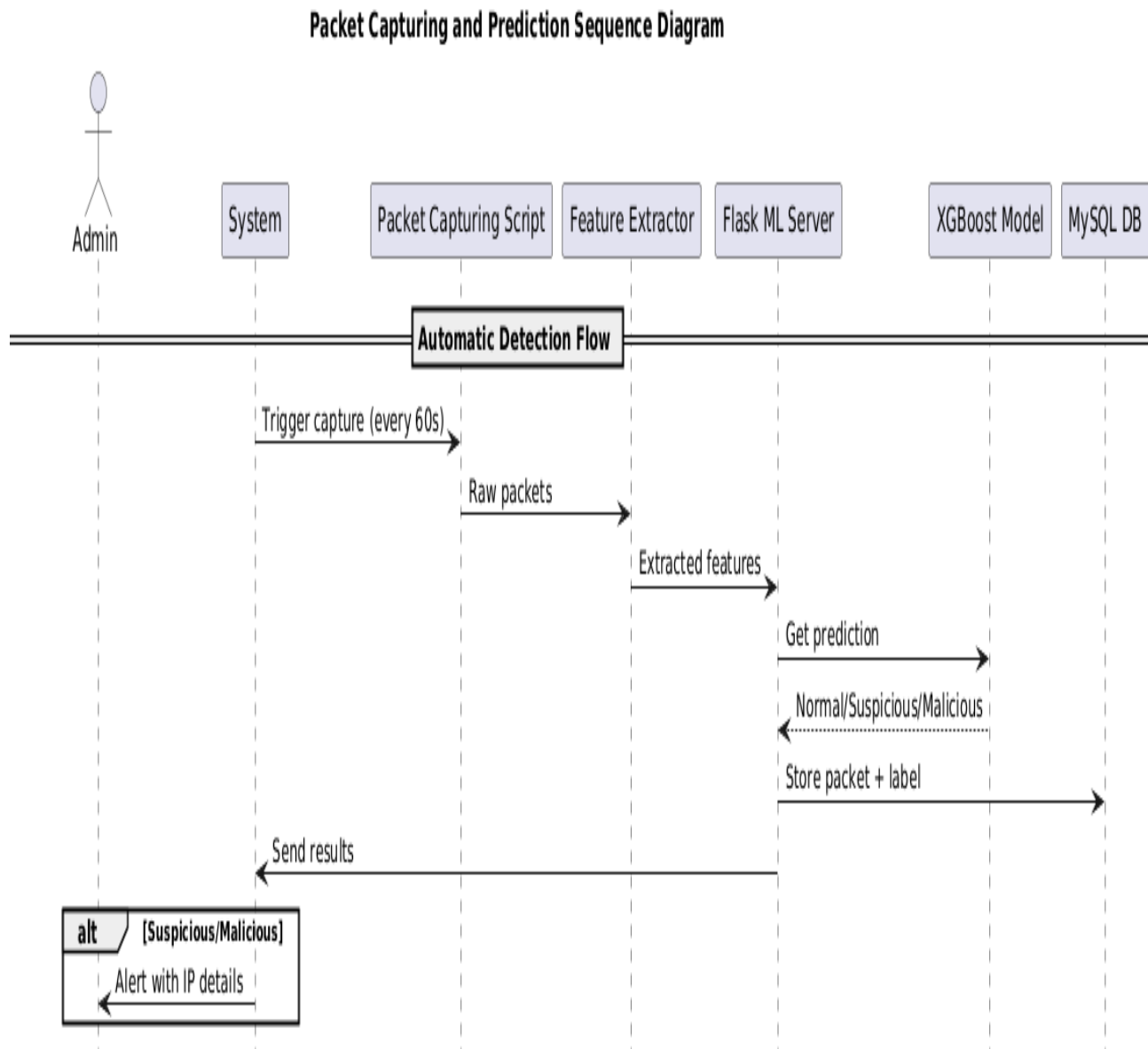
3.3.2 Packet Capture and Prediction Sequence Diagram

Actors:

- System (main controller)
- Packet Capturing Script
- Feature Extractor
- Flask ML Server
- ML Model (XGBoost)
- MySQL Database

Description:

- The System triggers the capturing process every 60 seconds.
- The Packet Capturing Script collects TCP and UDP packets from live network traffic.
- The collected packets are sent to the Feature Extractor.
- The Feature Extractor converts packets into flows and extracts 15 relevant features.
- The extracted data is forwarded to the Flask ML Server.
- The Flask Server sends the data to the XGBoost model for classification.
- The ML Model returns predictions: Normal, Suspicious, or Malicious.
- The Flask Server stores all packets and their labels in the MySQL Database.
- If any packet is suspicious/malicious, an alert is sent back to the System for admin notification.
- Admin manually blocks/unblocks IPs via System



3.3.2 Packet Capturing and Prediction Sequence Diagram

3.3.3. Admin Login and Authentication Sequence Diagram

Actors:

- Admin
- Web Browser
- Flask Server
- MySQL Database

Description:

- The Admin enters credentials on the Web Browser.
- The Browser sends a login request to the Flask Server.
- The Flask Server queries the MySQL Database to validate credentials.
- The Database responds with either success or failure.
- If credentials are valid:
 - The Server redirects the Admin to the dashboard.
- If invalid:
 - An error message is shown on the Browser.

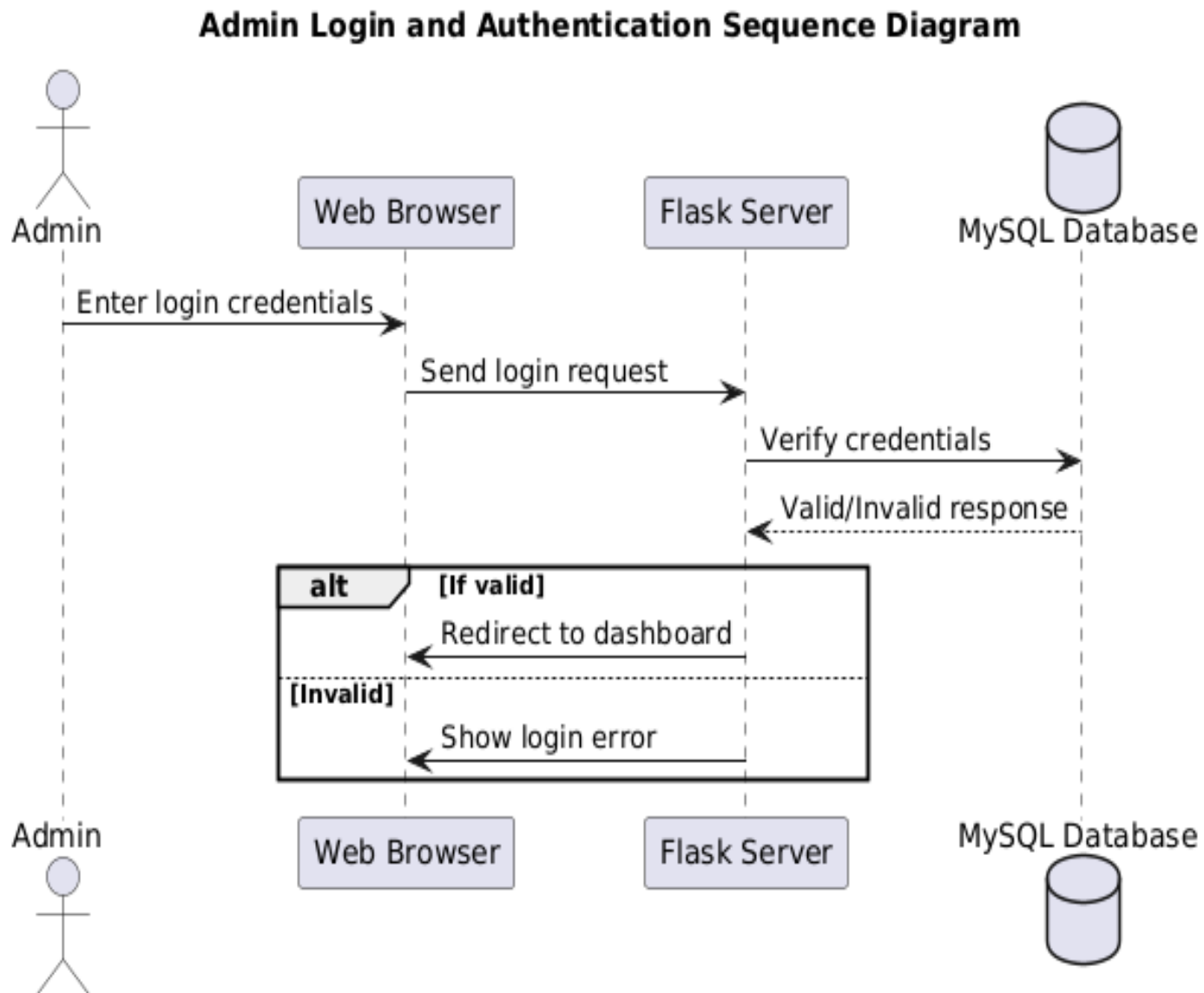


Fig 3.3.3 Admin Login and Authentication Sequence Diagram

3.3.4 Real-Time Alert Notification Sequence Diagram

Actors:

- Packet Capturing Script
- Flask Server
- WebSocket / Dashboard
- Admin

Description:

- The Packet Capturing Script sends prediction results to the Flask Server.
- If the result is *Suspicious* or *Malicious*:
 - The Flask Server sends a real-time alert to the WebSocket-connected Dashboard.
 - The Dashboard instantly shows the alert notification to the Admin.
- The Admin is kept informed of threats without delay.

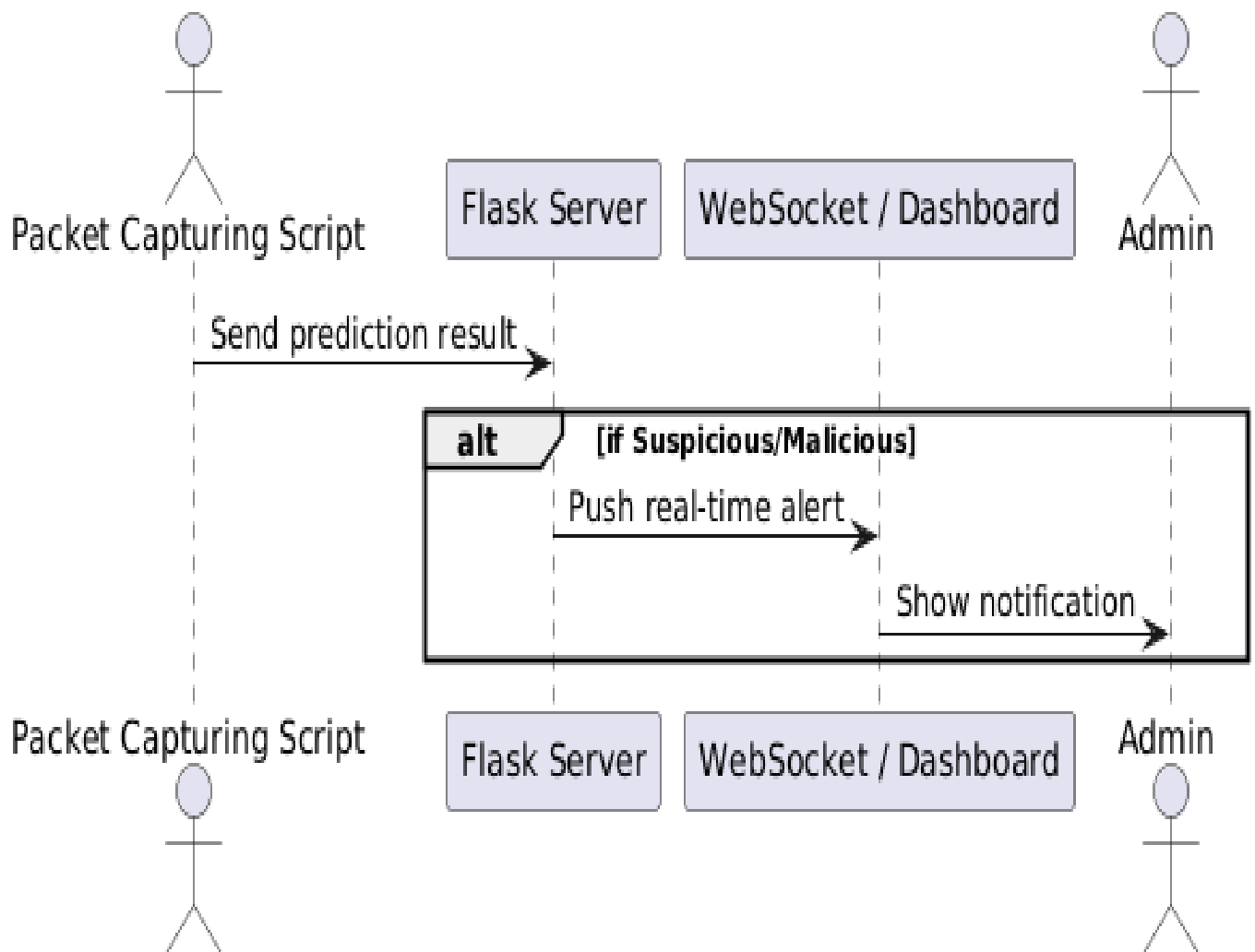
Real-Time Alert Notification Sequence Diagram

Fig 3.3.4 Real-Time Alert Notification Sequence Diagram

3.3.5 Admin Viewing Traffic Stats and Logs Sequence Diagram

Actors:

- Admin
- Web Browser
- Flask Server
- MySQL Database

Description:

- The Admin interacts with the Web Browser to request traffic data or packet logs.
- The Browser sends the request to the Flask Server.
- The Flask Server queries the MySQL Database for relevant statistics (e.g., TCP/UDP counts, top IPs).
- The Database sends the requested data to the Server.
- The Server renders the information on the Admin Dashboard for review.

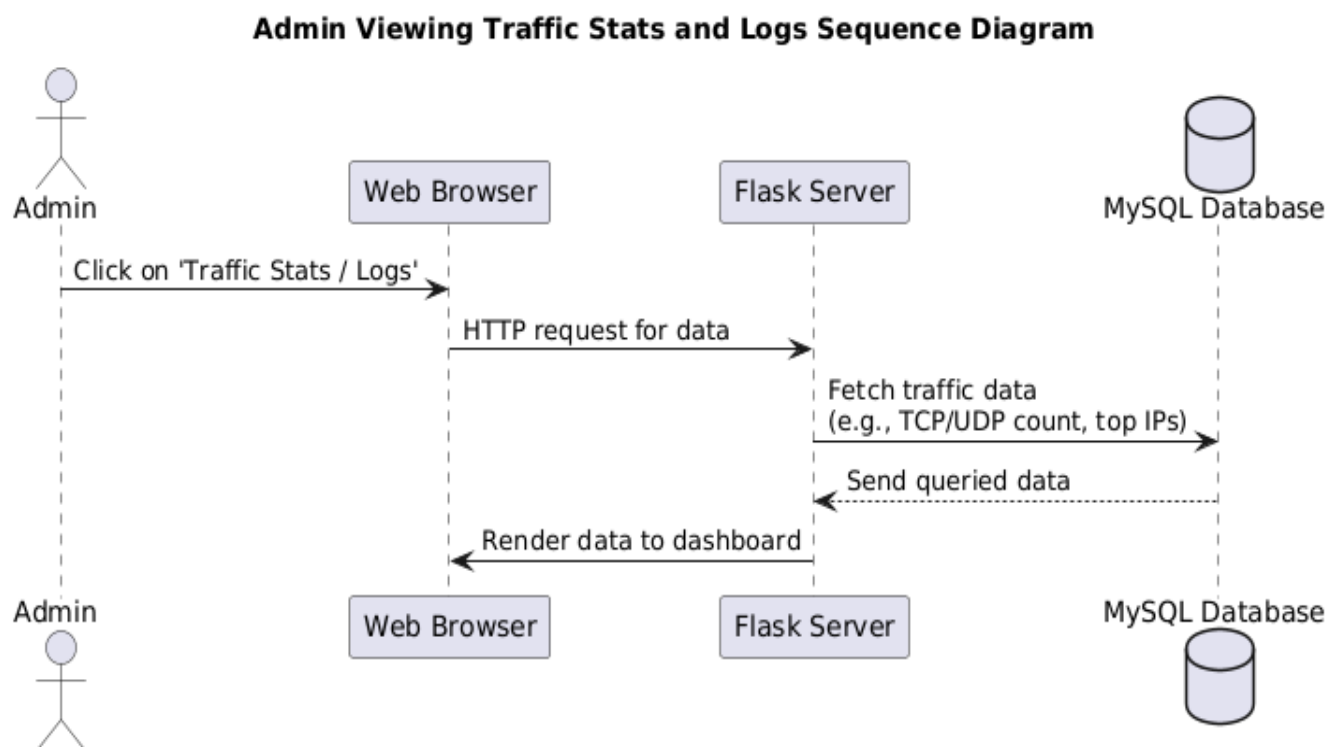


Fig 3.3.5 Admin Viewing Traffic Stats and Logs Sequence Diagram

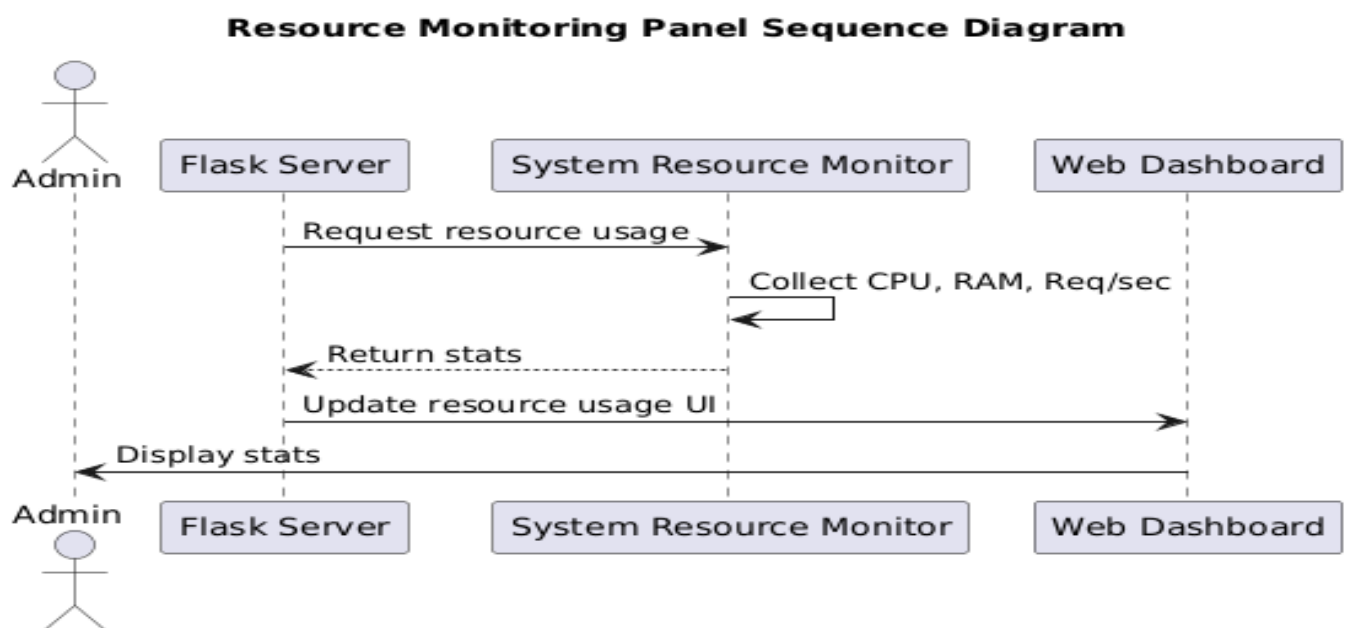
3.3.6 Resource Monitoring Panel Sequence Diagram

Actors:

- Admin
- Flask Server
- System Resource Monitor
- Web Dashboard

Description:

- The Flask Server periodically requests resource usage from the System Resource Monitor.
- The Resource Monitor collects:
 - CPU and memory usage of the packet capturing script
 - Requests per second
- The collected data is sent back to the Flask Server.
- The Server updates the Web Dashboard.
- The Admin sees real-time system performance data directly in the dashboard.



3.3.6 Resource Monitoring Panel Sequence Diagram

3.3.7 IP Blocking/Unblocking Sequence Diagram

Actors:

- Admin
- Web Dashboard
- System
- Firewall Controller
- MySQL Database

Description:

1. Admin clicks "IP Restrictions" in Web Dashboard
2. Admin sees block page with form
3. Admin submits:
 - IP address
 - Block duration
 - Reason
4. System receives data
5. System forwards to Firewall Controller
6. Firewall executes netsh advfirewall rule
7. Firewall confirms update to System
8. System logs in MySQL Database
9. Dashboard updates blocked IP list
10. For unblocking:
 - Admin clicks "Unblock" button
 - System removes firewall rule
 - Database updates status

- Dashboard refreshes list

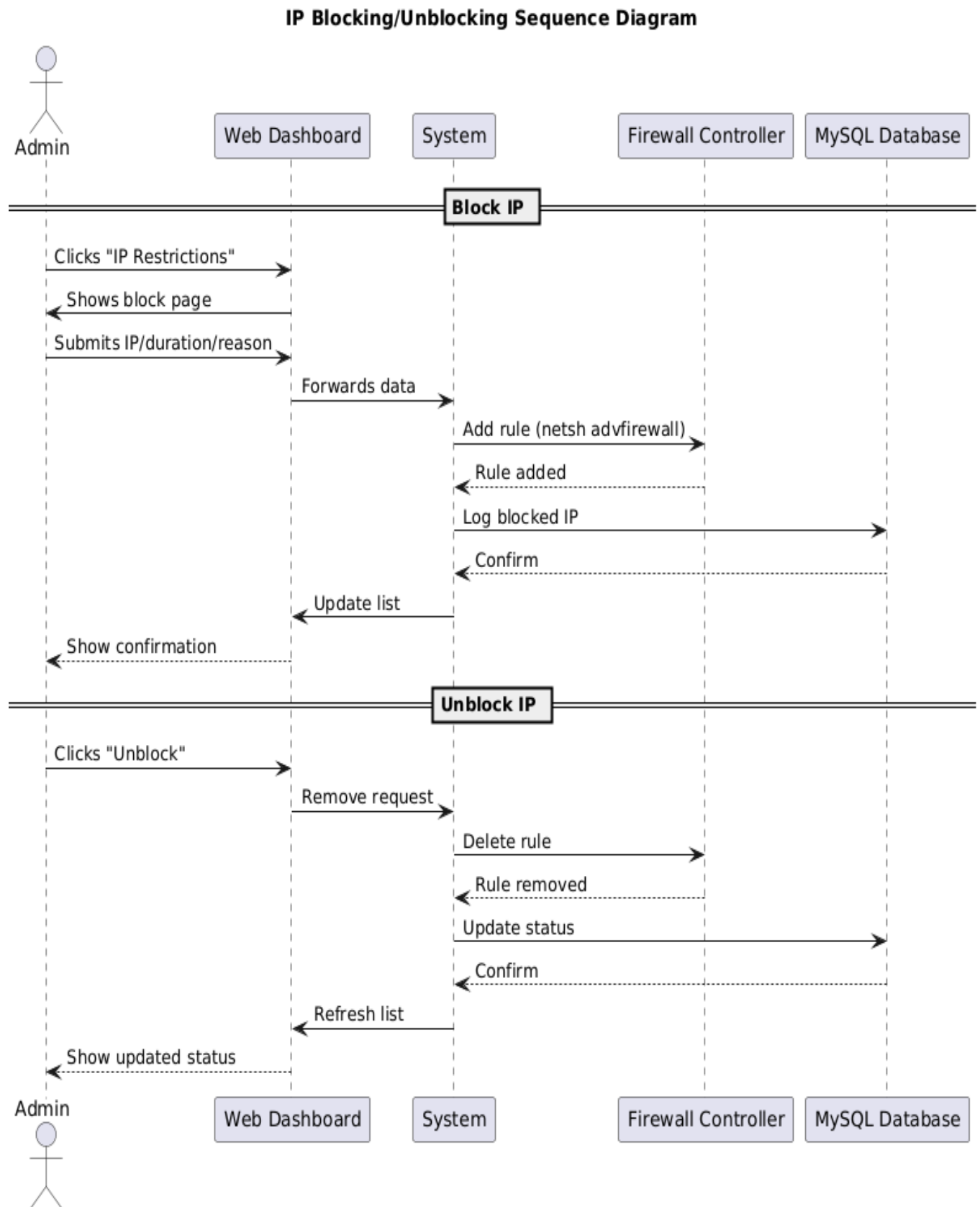


Fig 3.3.7 IP Blocking/Unblocking Sequence Diagram

3.4 Database Design

Introduction to MySQL

For the SmartNet real-time network traffic analyzer, MySQL is chosen for its robust performance, scalability, and strong support for transactional integrity. The system requires efficient storage and retrieval of network traffic data, user credentials, and analysis results, which makes MySQL an ideal choice. Data is normalized into two core tables, `admin_users` and `packet_data`, reducing redundancy while ensuring that the system can handle the continuous flow of network traffic and alert data effectively.

Data Modeling Approach

In SmartNet, we have the following key entities:

- **Admin Users:** Stores administrator credentials and profile information for system access and management.
- **Packet Data:** Stores detailed information about each network packet analyzed by the system, including features such as packet source and destination, protocol, timestamps, and machine learning classification results (normal, suspicious, or malicious).

Table Schemas

Admin Users Table

The `admin_users` table is responsible for storing user credentials and profile information for the administrators who manage the SmartNet panel. This includes fields like the administrator's email, hashed password, and profile information. The `id` field is a primary key, and email is uniquely indexed to ensure quick lookup.

 <code>admin_users</code>
<code>image: longblob</code>
<code><u>id</u>: int(11)</code>
<code><u>email</u>: varchar(255)</code>
<code>password_hash: varchar(255)</code>
<code>created_at: timestamp = current_timestamp()</code>
<code>name: varchar(25)</code>

Fig 3.4 (a) Admin_Users Table

Packet Data Table

The packet_data table stores the details of network traffic flows captured by SmartNet. For each flow, the table records information such as source and destination IP addresses, source and destination ports, protocol, various packet size metrics, and the machine learning label predicting whether the packet is normal, suspicious, or malicious. The id field serves as the primary key for unique identification of each flow record.

E	PacketData
○ id : int «PK» - source_port : int - destination_port : int - timestamp : double - bwd_packet_length_min : int - bwd_packets_per_s : double - bwd_packet_length_std : double - average_packet_size : double - packet_length_std : double - init_win_bytes_backward : int - psh_flag_count : int - fwd_packet_length_max : int - bwd_header_length : int - total_backward_packets : int - active_std : double - fwd_header_length : int - total_length_of_bwd_packets : int - max_packet_length : int - json_packet : longtext	
- source_ip : varchar(45) - destination_ip : varchar(45) - protocol : varchar(10) - direction : varchar(50) - label : varchar(20)	

Fig 3.4 (b) Packet Data Table

Blocked IPs Table

The blocked_ips table stores the details of all IP addresses manually blocked by SmartNet administrators. For each blocked IP, the table records the IP address, firewall rule name, timestamp of blocking, duration of the block (in seconds), expiration time (if temporary), and the reason for blocking. The id field serves as the primary key for unique identification of each blocking record.

E blocked_ips
- id: int «PK» - blocked_at: timestamp - expires_at: timestamp - duration_seconds: int - reason: text
ip_address: varchar(15) rule_name: varchar(100)

Fig 3.4 (c) Blocked IPs Table

3.5 Integration and Data Flow

User Authentication

- **Admin Users Table:** The administrator interacts with the SmartNet web panel. The authentication and session management of the administrator are managed through the admin_users table. The login process ensures that only authorized users can access and manage the system.

Network Packet Analysis and Data Capture

- **Packet Data Table:** The system captures TCP and UDP network packets every 60 seconds using a Python script with the tshark library. The captured data is processed to extract features, such as packet size, protocol, and packet lengths, and stored in the packet_data table.
- **Machine Learning Classification:** The extracted features are sent to a Flask server, which uses an XGBoost model to classify each packet as either normal, suspicious, or malicious based on the packet's characteristics. The result (label) is stored in the label field of the packet_data table.
- **IP Restriction and Blocking:** When the machine learning model flags traffic as malicious, the admin can review the associated IP addresses in the SmartNet web panel. If an IP is confirmed to be causing malicious traffic, the administrator can block it

through the IP Restrictions page. This action prevents further traffic from that IP, enhancing network security by mitigating ongoing threats.

Alerts and Notifications

- **Flask Server:** The server provides real-time predictions and alerts to the administrator based on packet analysis. If a suspicious or malicious packet is detected, an alert is sent to the admin on the SmartNet web panel. The data is then added to the MySQL database for further analysis.
- **Web Panel:** The panel provides a graphical interface where the administrator can view details such as the current packet status (normal, suspicious, or malicious), top IP addresses, total packet counts (TCP/UDP), and traffic rates. Additionally, the system displays resource usage, packet capture statistics, and machine learning model performance metrics. If the administrator identifies an IP address as a source of malicious traffic, he can block it using the IP Restrictions page in the panel.

Exporting Packet Data

- **Packet Data Download:** The administrator can download the entire dataset from the packet_data table in CSV format. This functionality allows the admin to export all captured packet data, including its features and classification labels, for offline analysis or reporting.

3.6 Entity Relationship Diagrams (ERDs)

In the SmartNet real-time network traffic analyzer system, the Entity Relationship Diagram (ERD) represents the entities involved in the system and the relationships between them. The three main entities are AdminUsers, PacketData, and BlockedIPs.

Entities:

1. AdminUsers:

Represents administrators who manage the system. Contains authentication fields (email, password_hash), profile data (name), and timestamps.

2. PacketData:

Stores network traffic data captured by SmartNet, including source/destination IPs, ports, protocol and ML classification labels (Normal/Suspicious/Malicious).

3. **BlockedIPs:**

Records manually blocked IP addresses with metadata:

- IP address, firewall rule name
- Block timestamps (initiated/expiration)
- Duration (seconds) and reason
- Linked to the admin who initiated the block

Relationships:

1. **AdminUsers** → **PacketData** (1-to-Many):

- One admin can manage multiple packet records.

2. **AdminUsers** → **BlockedIPs** (1-to-Many):

- One admin can block multiple IPs.

3. **BlockedIPs** → **PacketData** (1-to-Many):

- One blocked IP may relate to multiple malicious packets.

ERD Key Features:

- **Audit Trail:** BlockedIPs logs which admin blocked an IP and why.
- **Temporal Control:** Expiration timestamps enable automatic unblocking.
- **Threat Correlation:** Optional link between blocked IPs and malicious packets.

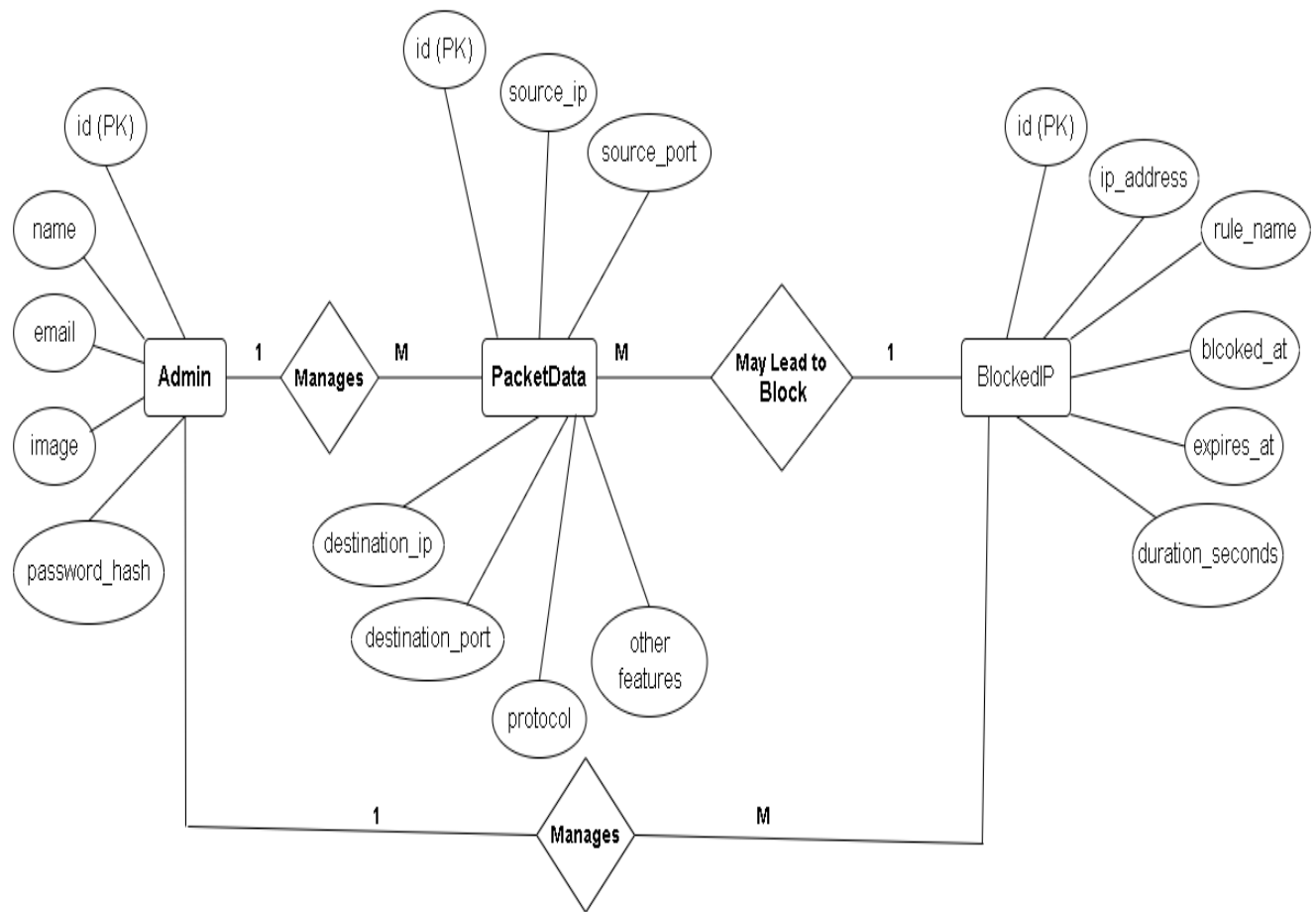


Fig 3.6 Entity Relationship Diagram

Description of the ERD

The Entity Relationship Diagram for the SmartNet system models the relationships between the AdminUsers, PacketData, and BlockedIPs entities. Below is a detailed breakdown:

AdminUsers Entity

The AdminUsers table stores administrator information:

- **id**: Unique identifier (Primary Key)
- **email**: Admin's email address
- **password_hash**: Hashed password for authentication
- **created_at**: Account creation timestamp
- **name**: Admin's name
- **image**: Profile picture

PacketData Entity

The PacketData table stores network flow statistics:

- **id**: Unique flow identifier (Primary Key)
- **source_ip, source_port**: Origin IP/port
- **destination_ip, destination_port**: Target IP/port
- **protocol**: TCP/UDP
- **timestamp**: Flow start time
- **direction**: Inbound/outbound
- **Flow metrics**: bwd_packet_length_min, bwd_packets_per_s, etc.
- **label**: ML classification (normal/suspicious/malicious)
- **json_packet**: All packets in the flow as JSON array

BlockedIPs Entity

The BlockedIPs table records manually blocked IPs:

- **id**: Unique identifier (Primary Key)
- **ip_address**: Blocked IP (e.g., "192.168.1.1")
- **rule_name**: Firewall rule identifier
- **blocked_at**: Timestamp of blocking
- **expires_at**: Optional expiration time
- **duration_seconds**: Block duration
- **reason**: Admin's justification

3.7 Building a Relational Model from ERDs

In the SmartNet real-time network traffic analyzer system, a single administrator has complete control over network monitoring, packet data management, and IP security controls. The MySQL database contains three core tables:

1. AdminUsers Table

Stores the exclusive administrator account:

- **Fields:** id (Primary Key), email, password_hash, created_at, name, image
- **Cardinality:** Exactly one record (singleton design)
- **Purpose:** Manages the sole admin's authentication and profile.

2. PacketData Table

Stores aggregated network flow statistics:

- **Key Fields:**
 - Identifiers: source_ip, destination_ip, source_port, destination_port
 - Metadata: protocol, timestamp, direction
 - Behavioral metrics: bwd_packet_length_min, bwd_packets_per_s, etc.
 - Classification: label (Normal/Suspicious/Malicious)
 - Raw data: json_packet (complete flow packets in JSON)

3. BlockedIPs Table

Manages firewall-level IP restrictions:

- **Fields:**
 - ip_address: Blocked IPv4 address
 - rule_name: Firewall rule identifier
 - Temporal data: blocked_at, expires_at, duration_seconds
 - Context: reason

Relationships

1. AdminUsers → PacketData (1-to-Many)

- Notation: AdminUsers "1" -- "0..*" PacketData : manages >
- **Semantics:** The admin oversees all network flows.

2. AdminUsers → BlockedIPs (1-to-Many)

- Notation: AdminUsers "1" -- "0..*" BlockedIPs : blocks >
- **Semantics:** The admin enforces all IP blocks.

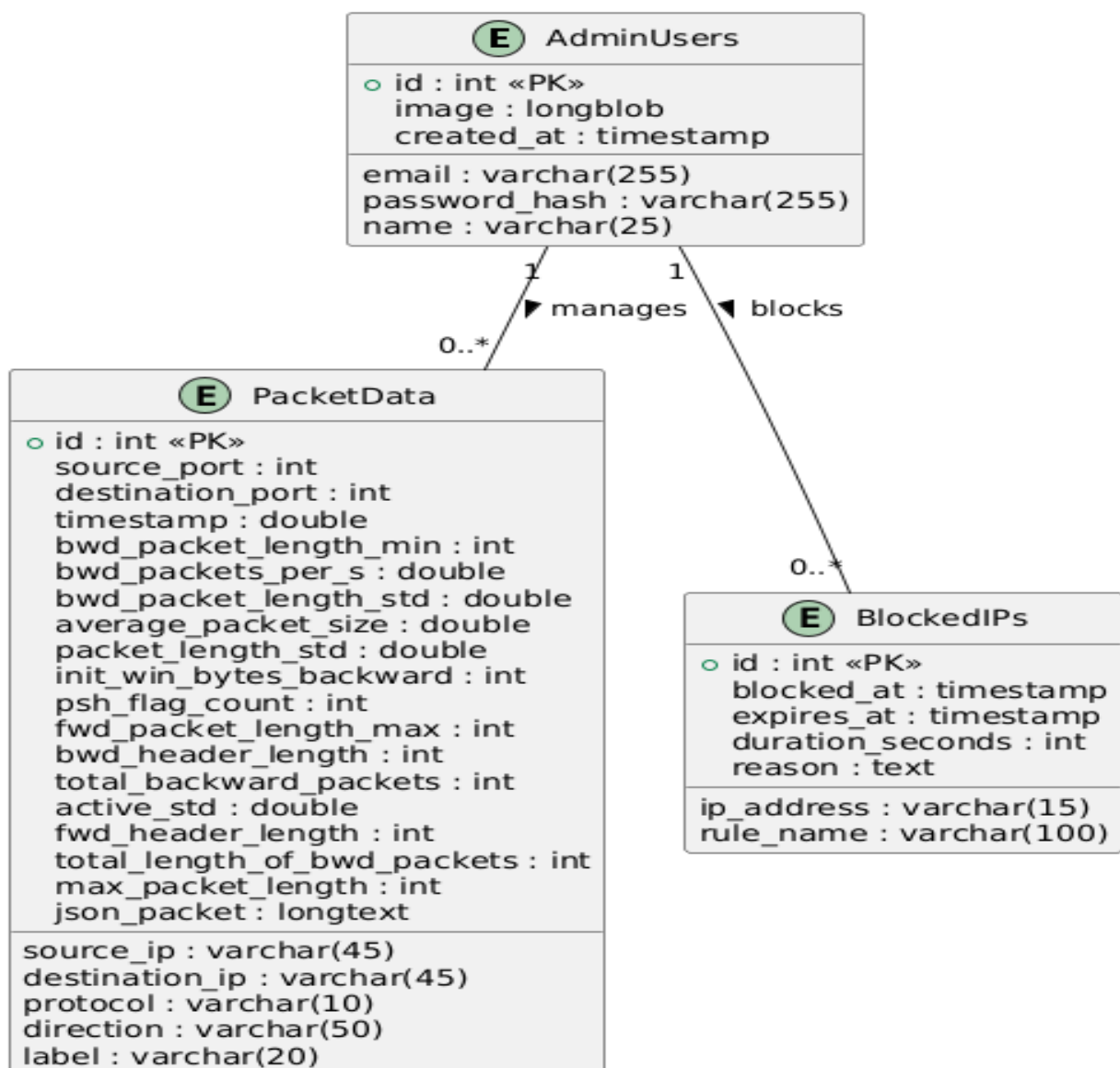


Fig 3.7 Relational Model from ERD

Chapter 4

System Development & Implementation

4.1 Introduction to System Development

The development of the Smartnet Real-Time Network Traffic Analyzer was carried out using a structured and modular approach to ensure scalability, maintainability, and efficiency. This chapter provides an in-depth overview of the tools, technologies, and methodologies employed during the system development and implementation phase of the project.

System development began with the detailed design of the architecture, which included traffic capturing, machine learning-based classification, database management, and web interface functionalities. The process was broken down into clearly defined phases, including system design, data collection and preprocessing, machine learning model training, backend development using Python and Flask, and frontend interface creation for the admin panel.

The primary objective of the development phase was to integrate real-time packet capturing with intelligent classification to detect suspicious or malicious traffic. The system was designed to be responsive, user-friendly, and efficient in analyzing network flows while maintaining accuracy through the use of a pre-trained XGBoost model. Furthermore, special focus was placed on ensuring that alerts are generated immediately upon detection of abnormal traffic and that all data is stored securely in a MySQL database for further review.

By leveraging modern technologies such as Flask, XGBoost, TShark, NumPy, and MySQL, the system ensures high performance and real-time data processing. This chapter elaborates on each component developed, the rationale behind its design, and the integration approach followed to achieve a fully functional and responsive system.

4.2 Tool/Language Selection

The selection of appropriate tools, programming languages, and libraries was a critical step in the successful development and implementation of the Smartnet Real-Time Network Traffic Analyzer. The tools were chosen based on their performance, compatibility, ease of use, and ability to handle real-time data processing and machine learning tasks efficiently.

Below is a detailed justification of the main tools and technologies used:

- **Python:** Python was selected as the core programming language due to its simplicity, readability, and extensive support for data science, networking, and backend development. It also provides powerful libraries for machine learning and packet manipulation.

- **TShark:** TShark, the command-line version of Wireshark, was used to capture live TCP and UDP packets from the network. It is lightweight, supports extensive filtering, and integrates smoothly with Python using subprocess modules.
- **XGBoost:** XGBoost (Extreme Gradient Boosting) was chosen for its high accuracy, speed, and performance in machine learning tasks. It was used to train a classifier that labels packets as normal, suspicious, or malicious based on extracted features.
- **NumPy:** NumPy was utilized to convert incoming feature vectors into numerical arrays, which are required for making predictions using the XGBoost model.
- **Flask:** Flask, a lightweight Python web framework, was used to build the backend server and REST API for handling predictions, database interactions, and hosting the admin panel.
- **MySQL:** MySQL was chosen as the database management system due to its reliability, performance, and ease of integration with Python applications. It was used to store both admin data and all packet-related records.
- **HTML/CSS/JavaScript:** These standard web technologies were used to build the front-end admin panel, providing the user with an interactive and responsive interface to monitor threats and manage the system.

Each of these tools played a significant role in achieving the system's goals, ranging from data collection and classification to database storage and user interaction. The combined use of these technologies ensured that the system remains lightweight, accurate, and capable of performing in real-time environments.

4.3 Hardware for the System

The development, training, testing, and deployment of the Smartnet Real-Time Network Traffic Analyzer system were carried out on a personal laptop with the following hardware specifications:

- **System Model:** Dell Vostro 14-3468
- **Processor:** Intel Core i7, 7th Generation
- **Cores/Threads:** 1 Physical Processor with 4 Logical Processors

- **RAM:** 16 GB
- **Primary Storage (SSD):** 256 GB (Windows 11 Pro Installed)
- **Secondary Storage (HDD):** 500 GB
- **Operating System:** Windows 11 Pro (64-bit)

This configuration provided sufficient performance for handling tasks such as live packet capturing using TShark, real-time predictions using the trained XGBoost model, and running the Flask-based web server alongside a MySQL database. The 16 GB of RAM allowed smooth multitasking and efficient memory handling, especially during machine learning inference and real-time data processing.

Although the system was developed and tested locally on this hardware setup, the overall architecture is scalable and can be deployed on more powerful servers or cloud platforms if needed in the future.

4.4 Software Development & Implementation

This section describes the technologies and tools used in the development of both the client-side and server-side components of the Smartnet Real-Time Network Traffic Analyzer system.

4.4.1 Client-Side Technology

The client-side of the system was developed using standard web technologies including HTML, CSS, JavaScript and the Bootstrap.

- **HTML** was used to define the structure of web pages such as the admin dashboard, login screen, profile management, and data visualization panels.
- **CSS** was applied to style and format the interface for a clean and professional look.
- **JavaScript** was used to add interactivity, fetch real-time updates from the backend (using AJAX), and dynamically display charts for traffic statistics.
- **Bootstrap**, a popular front-end framework, was used extensively to design a responsive, mobile-friendly, and visually appealing UI. It provided ready-to-use components such as navigation bars, tables, alerts, buttons, and grid layouts, which sped up development and ensured design consistency.

Additionally, chart libraries like Chart.js were utilized to visualize data such as traffic rates, packet distribution, and threat levels.

4.4.2 Server-Side Technology

The server side of the system was developed using Python, with Flask as the web framework. Flask enabled rapid development of RESTful routes to receive packet data, perform predictions using the pre-trained XGBoost model, and send real-time alerts to the front-end.

- Python scripts captured packets using the TShark library, extracted relevant features, and forwarded them to the Flask server.
- The server used NumPy and scikit-learn for preprocessing and model inference.
- The MySQL Connector for Python was used for interacting with the database where all packet logs were stored.
- Flask also served the HTML pages and handled admin login functionalities securely.

The use of Flask allowed the backend to remain lightweight yet efficient, capable of handling real-time traffic and dynamic web content delivery.

4.5 Code/Algorithms of Important Modules

This section highlights the key algorithms and code snippets used in the system's core modules.

4.5.1 XGBOOST Model For Packets Prediction

```
#!/usr/bin/env python

# coding: utf-8

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score, confusion_matrix,
classification_report, average_precision_score,
```

```
precision_recall_curve)

from xgboost import XGBClassifier

import xgboost as xgb

import matplotlib.pyplot as plt

import pickle

import json

df = pd.read_csv('unified_data4.csv')

df = df.dropna()

df.reset_index(drop=True, inplace=True)

df_X = df.drop(['Label'], axis='columns')

df_Y = df['Label']

inf_mask = np.isinf(df_X).any(axis=1)

df_X = df_X[~inf_mask]

df_X.reset_index(drop=True, inplace=True)

df_Y = df_Y[~inf_mask]

df_Y.reset_index(drop=True, inplace=True)

df_Y = np.where(df_Y == 'BENIGN', 0, 1)

df_Y = pd.DataFrame(df_Y, columns=['Label'])

xgb_model = xgb.XGBClassifier(n_estimators=1000, booster='gbtree', random_state=42)

xgb_model.fit(df_X, df_Y)

feature_importance = xgb_model.feature_importances_

importance_df = pd.DataFrame({"Feature": df_X.columns, "Importance":
feature_importance})

importance_df = importance_df.sort_values(by="Importance", ascending=False)
```

```
top_15_features = importance_df.head(15)["Feature"].tolist()

df_X_top15 = df_X[top_15_features]

df_combined = pd.concat([df_X_top15, df_Y], axis=1)

df_combined_no_Nan = df_combined.fillna(df_combined.median())

X = df_combined_no_Nan.drop(['Label'], axis=1)

y = df_combined_no_Nan['Label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

xgb_model_selected4 = xgb.XGBClassifier(

n_estimators=1000,

booster='gbtree',

random_state=42,

eval_metric='logloss',

use_label_encoder=False

)

xgb_model_selected4.fit(X_train, y_train)

y_pred = xgb_model_selected4.predict(X_test)

y_proba = xgb_model_selected4.predict_proba(X_test)[:, 1]

metrics = {

'Accuracy': accuracy_score(y_test, y_pred),

'Precision': precision_score(y_test, y_pred),

'Recall': recall_score(y_test, y_pred),

'F1 Score': f1_score(y_test, y_pred),

'ROC AUC': roc_auc_score(y_test, y_proba),

'Average Precision': average_precision_score(y_test, y_proba)
```

```
}

print("\n" + "="*40)

print("CLASSIFICATION METRICS")

print("="*40)

print(pd.DataFrame.from_dict(metrics, orient='index', columns=['Score']).round(4))

print("\nCONFUSION MATRIX:")

print(confusion_matrix(y_test, y_pred))

print("\nCLASSIFICATION REPORT:")

print(classification_report(y_test, y_pred))

with open("xgb_model_selected4.pkl", "wb") as f:

    pickle.dump(xgb_model_selected4, f)

columns = {'data_columns': [col for col in X.columns]}

with open("columns.json", "w") as f:

    json.dump(columns, f)

print("\n✅ Model and artifacts saved successfully:")

print("- xgb_model_selected4.pkl (model)")

print("- columns.json (feature names)")
```

4.5.2 Main Module for Capturing Packets and Handling Tasks

```
import sys

import io

import gc

import time

import threading

from network_utils import get_local_ip
```

```
from packet_processing import capture_packets

from send_to_server import send_data_to_server, send_laptop_ip_to_server

# Force unbuffered output

sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', line_buffering=True)

def async_send(data, url):

    """Send packet data to server in background"""

    print(f"Type of data to send {len(data)}")

    send_data_to_server(data, url)

def capture_and_classify():

    init_win_bytes_backward = {}

    gc.collect()

    laptop_ip = get_local_ip()

    if not laptop_ip:

        print("Failed to get laptop IP. Exiting...", flush=True)

        return

    print(f"Your Laptop's IP: {laptop_ip}", flush=True)

    # Send laptop IP to server first

    if not send_laptop_ip_to_server(laptop_ip):

        print("Failed to send laptop IP to server", flush=True)

        return

    try:

        while True:

            print("Capturing packets for 20 seconds...", flush=True)

            flows, total_packets = capture_packets(laptop_ip, capture_duration=10)
```

```
print(f"[DEBUG] In main: total_packets = {total_packets}, flows = {len(flows)}",
flush=True)

print("Flows 4 = ", len(flows))

print(f"Packets captured and now sending... (Total Packets: {total_packets})",
flush=True)

print(f"[DEBUG] In main: total_packets = {total_packets}, flows = {len(flows)}",
flush=True)

server_url = "http://127.0.0.1:5000/api/send_data"

if total_packets > 0:

    sender_thread = threading.Thread(target=async_send, args=(flows, server_url))

    sender_thread.start()

    print(f"🚀 Sending {len(flows)} packets in background...", flush=True)

else:

    print("No packets to send", flush=True)

print(f"\nTotal Unique Flows Captured: {len(flows)}", flush=True)

print(f"Total Packets Captured: {total_packets}\n", flush=True)

del flows, total_packets

gc.collect()

time.sleep(5)

print("Starting the next capture cycle...\n", flush=True)

except KeyboardInterrupt:

    print("\nProcess Interrupted. Exiting.....", flush=True)

    sys.exit(0)

if __name__ == "__main__":

    try:
```

```

        capture_and_classify()

    except KeyboardInterrupt:

        print("\nProcess Interrupted. Exiting.....")

```

4.5.3 Flask Server Function for Making Predictions

```

def prediction_worker():

    """Process packets in batches, make predictions, and send to DB queue"""

    global threat_count

    while True:

        try:

            batch_id, batch_data = packet_queue.get()

            if batch_id is None: # Graceful shutdown signal

                print("[ 🛑 ] Prediction worker received shutdown signal")

                break

            print(f"\n[ 🔍 ] Processing batch {batch_id} ({len(batch_data)} packets)")

            results = []

            for packet in batch_data:

                try:

                    # ===== 1. Feature Extraction =====

                    feature_vector = [

                        float(packet.get("Bwd Packet Length Min", 0.0)),

                        float(packet.get("Bwd Packets/s", 0.0)),

                        float(packet.get("Bwd Packet Length Std", 0.0)),

                        float(packet.get("Average Packet Size", 0.0)),

                        float(packet.get("Packet Length Std", 0.0)),

```



```

int(float(packet.get("Init_Win_bytes_backward", 0))),
int(float(packet.get("PSH Flag Count", 0))),
float(packet.get("Fwd Packet Length Max", 0.0)),
int(float(packet.get("Bwd Header Length", 0))),
int(float(packet.get("Destination Port", 0))),
int(float(packet.get("Total Backward Packets", 0))),
float(packet.get("Active Std", 0.0)),
int(float(packet.get("Fwd Header Length", 0))),
int(float(packet.get("Total Length of Bwd Packets", 0))),
int(float(packet.get("Max Packet Length", 0)))
]

# ===== 2. Prepare Model Input =====

features_array = np.array([feature_vector], dtype=np.float32)

features_array = np.around(features_array, decimals=6)

if features_array.shape != (1, 15):
    raise ValueError(f"Invalid feature shape {features_array.shape}, expected (1, 15)")

# ===== 3. Make Prediction =====

prediction = util.model.predict(features_array)[0]

proba = util.model.predict_proba(features_array)[0]

# Calculate probabilities

proba_normal = proba[0] * 100

proba_malicious = proba[1] * 100

proba_suspicious = max(0.0, 100 - (proba_normal + proba_malicious))

# ===== 4. Determine Verdict =====

```

```
if proba_malicious > 70:

    verdict = "Malicious"

    status = "malicious"

    # with threat_count_lock:

    threat_count += 1

    socketio.emit("packet_status", {

        "message": f"[Batch {batch_id}] from {packet['Source IP']} is Malicious",

        "src_ip": packet["Source IP"],

        "status": "suspicious"})

elif proba_normal > 70:

    verdict = "Normal"

    status = "normal"

else:

    verdict = "Suspicious"

    status = "suspicious"

    # with threat_count_lock:

    threat_count += 1

    socketio.emit("packet_status", {

        "message": f"[Batch {batch_id}] from {packet['Source IP']} is Suspicious",

        "src_ip": packet["Source IP"],

        "status": "suspicious"})

# ===== 7. Prepare Result =====

result = {

    **packet,
```

```
"prediction": int(prediction),
"confidence": float(max(proba)),
"label": verdict,
"proba_normal": proba_normal,
"proba_malicious": proba_malicious,
"proba_suspicious": proba_suspicious
}

results.append(result)

update_protocol_counts(packet)

except Exception as e:

    print(f"[ ❌ ] Packet processing error: {str(e)}", flush=True)

    print(f"Problematic packet: {packet}", flush=True)

    continue

# ===== 8. Send to Database Queue =====

db_queue.put((batch_id, results))

packet_queue.task_done()

print(f"[ ✅ ] Batch {batch_id} completed. Threats detected: {threat_count}", flush=True)

except Exception as e:

    print(f"[ 🔥 ] Critical prediction worker error: {str(e)}", flush=True)

    packet_queue.task_done() # Ensure queue doesn't block on error
```

4.5.4 Flask Server Function for Saving Packets to Database

```
def save_batch_to_db(batch_id, packets):

    """Insert a batch of packets into the database"""

    connection = None

    try:

        connection = create_db_connection()

        cursor = connection.cursor()

        # EXACT column list matching your table structure

        columns = [

            'source_ip', 'source_port', 'destination_ip', 'destination_port',

            'protocol', 'timestamp', 'direction',

            'bwd_packet_length_min', 'bwd_packets_per_s', 'bwd_packet_length_std',

            'average_packet_size', 'packet_length_std', 'init_win_bytes_backward',

            'psh_flag_count', 'fwd_packet_length_max', 'bwd_header_length',

            'total_backward_packets', 'active_std', 'fwd_header_length',

            'total_length_of_bwd_packets', 'max_packet_length', 'label', 'json_packet'

        ]

        # Parameterized SQL with exact column names

        sql = f"""

        INSERT INTO packet_data ({','.join(columns)})

        VALUES ({','.join(['%s']*len(columns))})

        """

        records = []

        for pkt in packets:
```

```
# Convert timestamp

timestamp = pkt.get("Timestamp", "")

if isinstance(timestamp, (int, float)):

    timestamp_unix = float(timestamp)

elif isinstance(timestamp, str) and timestamp:

    try:

        timestamp_obj = time.strptime(timestamp, "%Y-%m-%d %H:%M:%S")

        timestamp_unix = time.mktime(timestamp_obj)

    except ValueError:

        timestamp_unix = 0.0

    else:

        timestamp_unix = 0.0

# Map packet data to column order

record = (

    pkt.get("Source IP", "Unknown"),

    int(pkt.get("Source Port", 0)),

    pkt.get("Destination IP", "Unknown"),

    int(pkt.get("Destination Port", 0)),

    pkt.get("Protocol", "UNKNOWN"),

    timestamp_unix,

    pkt.get("Direction", "Unknown"),

    int(pkt.get("Bwd Packet Length Min", 0)),

    float(pkt.get("Bwd Packets/s", 0.0)),

    float(pkt.get("Bwd Packet Length Std", 0.0)),
```

```
float(pkt.get("Average Packet Size", 0.0)),
float(pkt.get("Packet Length Std", 0.0)),
int(pkt.get("Init_Win_bytes_backward", 0)),
int(pkt.get("PSH Flag Count", 0)),
int(pkt.get("Fwd Packet Length Max", 0)),
int(pkt.get("Bwd Header Length", 0)),
int(pkt.get("Total Backward Packets", 0)),
float(pkt.get("Active Std", 0.0)),
int(pkt.get("Fwd Header Length", 0)),
int(pkt.get("Total Length of Bwd Packets", 0)),
int(pkt.get("Max Packet Length", 0)),
pkt.get("label", "Unknown"),
pkt.get("json_packet", None)
)

# Debug: Verify value count matches columns
if len(record) != len(columns):
    print(f"[ ✖ ] Mismatch in record {len(record)} vs columns {len(columns)}")
    print("Record values:", record)
    continue
records.append(record)

# Execute with parameter verification
try:
    cursor.executemany(sql, records)
    connection.commit()
```

```
print(f"[✅ DB] Batch {batch_id} inserted ({len(records)} packets)")

except Exception as e:

print(f"[❌ SQL Execution Error] {str(e)}")

print("First record values:", records[0] if records else "No records")

print("SQL:", sql)

raise

except Exception as e:

print(f"[❌ DB Error] Batch {batch_id}: {str(e)}")

if connection:

connection.rollback()

finally:

if connection:

connection.close()
```

Chapter 5

User's Guide

5.1 Input Forms

This section provides a detailed overview of the system's user interface, including forms for user interaction and sample reports generated by the SmartNet: Realtime Network traffic Analyzer.. Screenshots of key functionalities are included to guide users through the process.

5.1.1 Admin Authentication Form

- **Description:** Admin enters his credentials (email and password) to access the system.
- **Fields:**
 - **Email** (required)
 - **Password** (required)
- **Screenshot:**

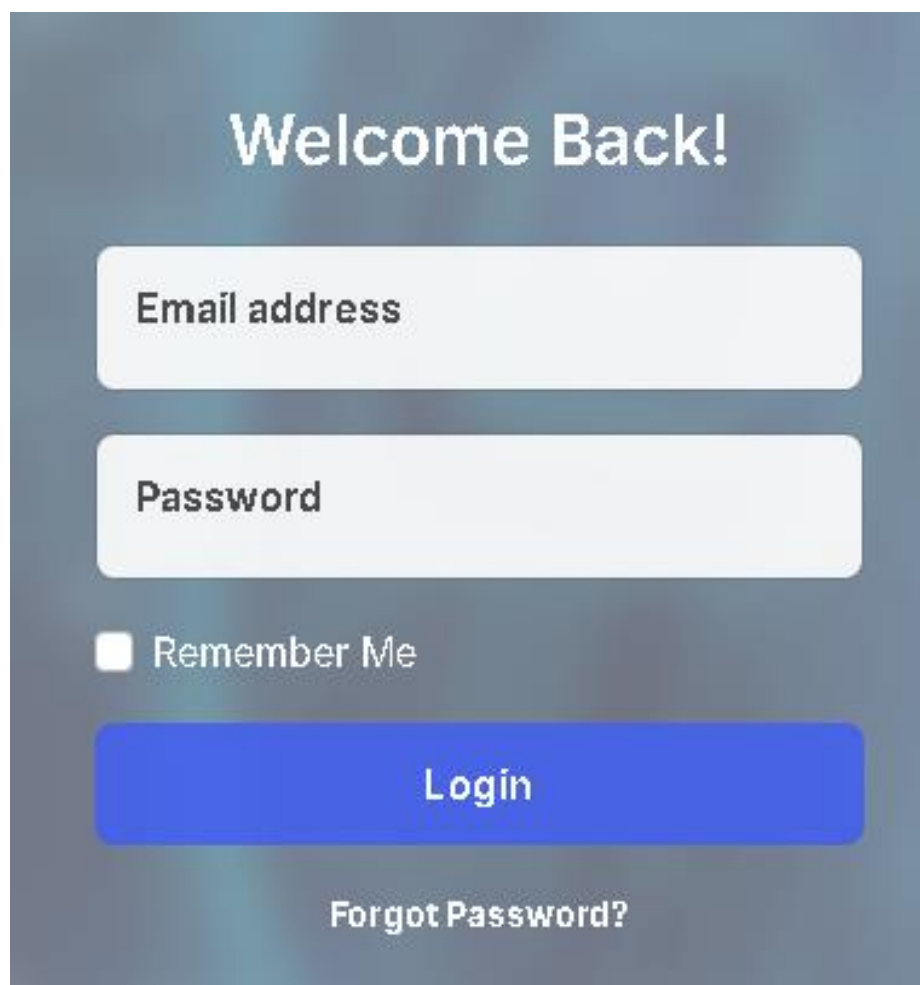
A screenshot of the Admin Authentication Form. The form has a dark blue background with white text. At the top, it says "Welcome Back!". Below this are two white input fields with rounded corners. The first field is labeled "Email address" and the second is labeled "Password". Below the password field is a checkbox labeled "Remember Me". At the bottom of the form is a large blue button with white text that says "Login". Below the button is a link that says "Forgot Password?".

Fig 5.1.1 Admin Authentication Form

5.1.2 Password Recovery Form

• **Description:** Allows administrators to reset their password when forgotten, using a three-step verification process:

1. Email verification
2. Code confirmation
3. New password setup

• **Fields:**

1. **Email** (required)
 - Valid email address registered in the system
 - Receives 6-digit verification code
2. **Verification Code** (required in step 2)
 - 6-digit numeric code sent via email
 - Valid for 10 minutes
 - Maximum 3 attempts allowed
3. **New Password** (required in step 3)
 - Minimum 8 characters
 - Must match confirmation field
 - Immediately invalidates all active sessions
4. **Confirm New Password** (required in step 3)
 - Must match new password exactly

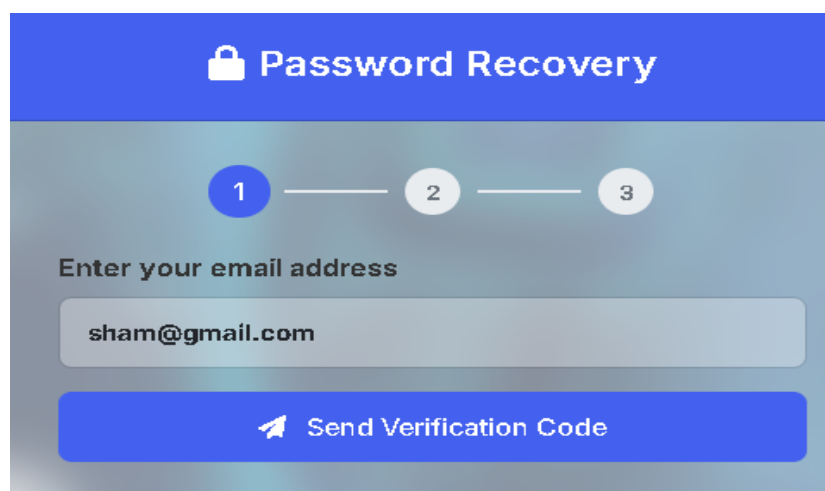
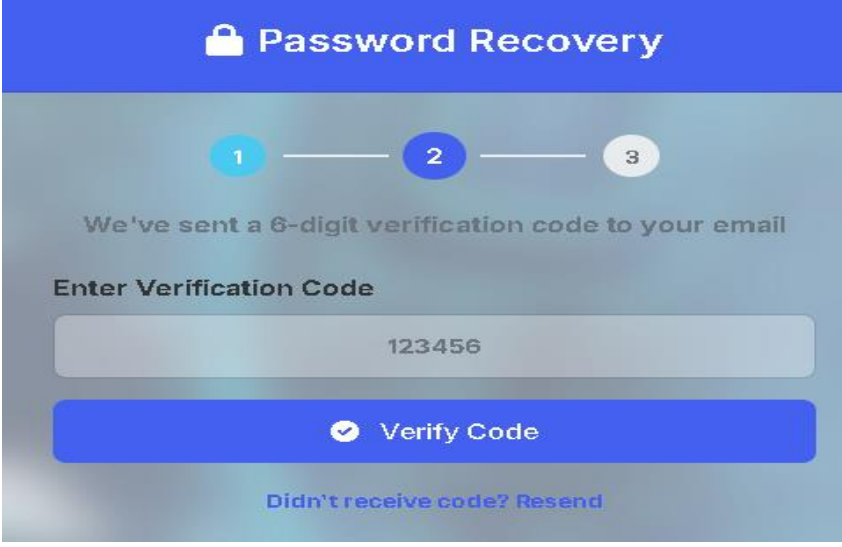
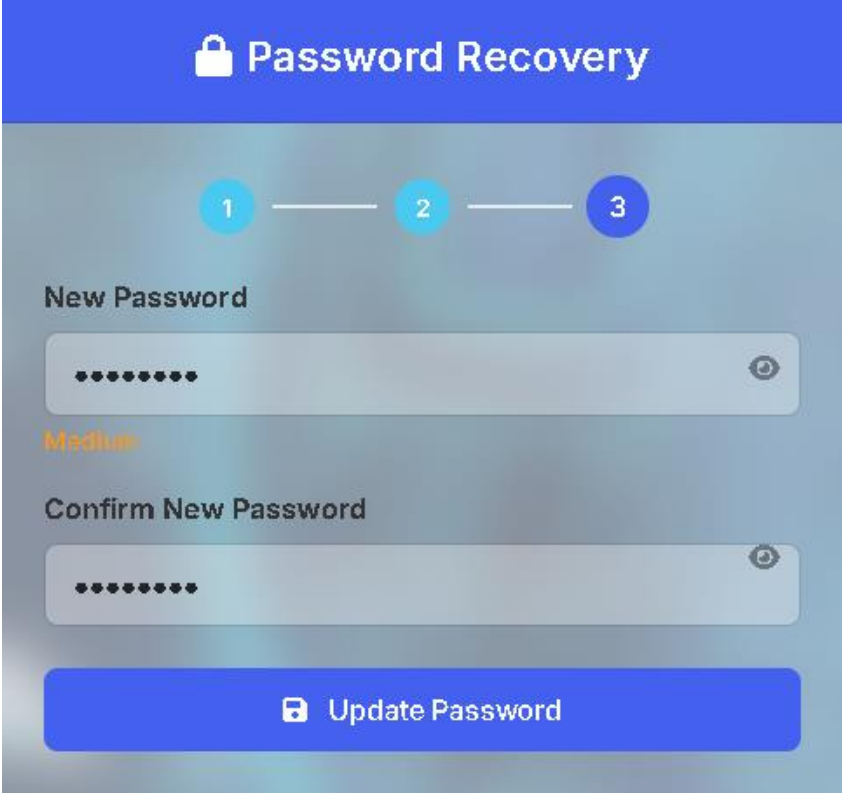


Fig 5.1.2 (a) Password Recovery Form



The image shows a mobile app interface for password recovery. At the top is a blue header with a white padlock icon and the text "Password Recovery". Below the header is a progress indicator with three circles: the first is light blue with the number "1", the second is dark blue with the number "2", and the third is light gray with the number "3". Below the progress indicator, the text "We've sent a 6-digit verification code to your email" is displayed. Underneath, the label "Enter Verification Code" is followed by a text input field containing the code "123456". Below the input field is a large blue button with a white checkmark icon and the text "Verify Code". At the bottom, there is a link that says "Didn't receive code? Resend" in blue text.

Fig 5.1.2 (b) Password Recovery Form

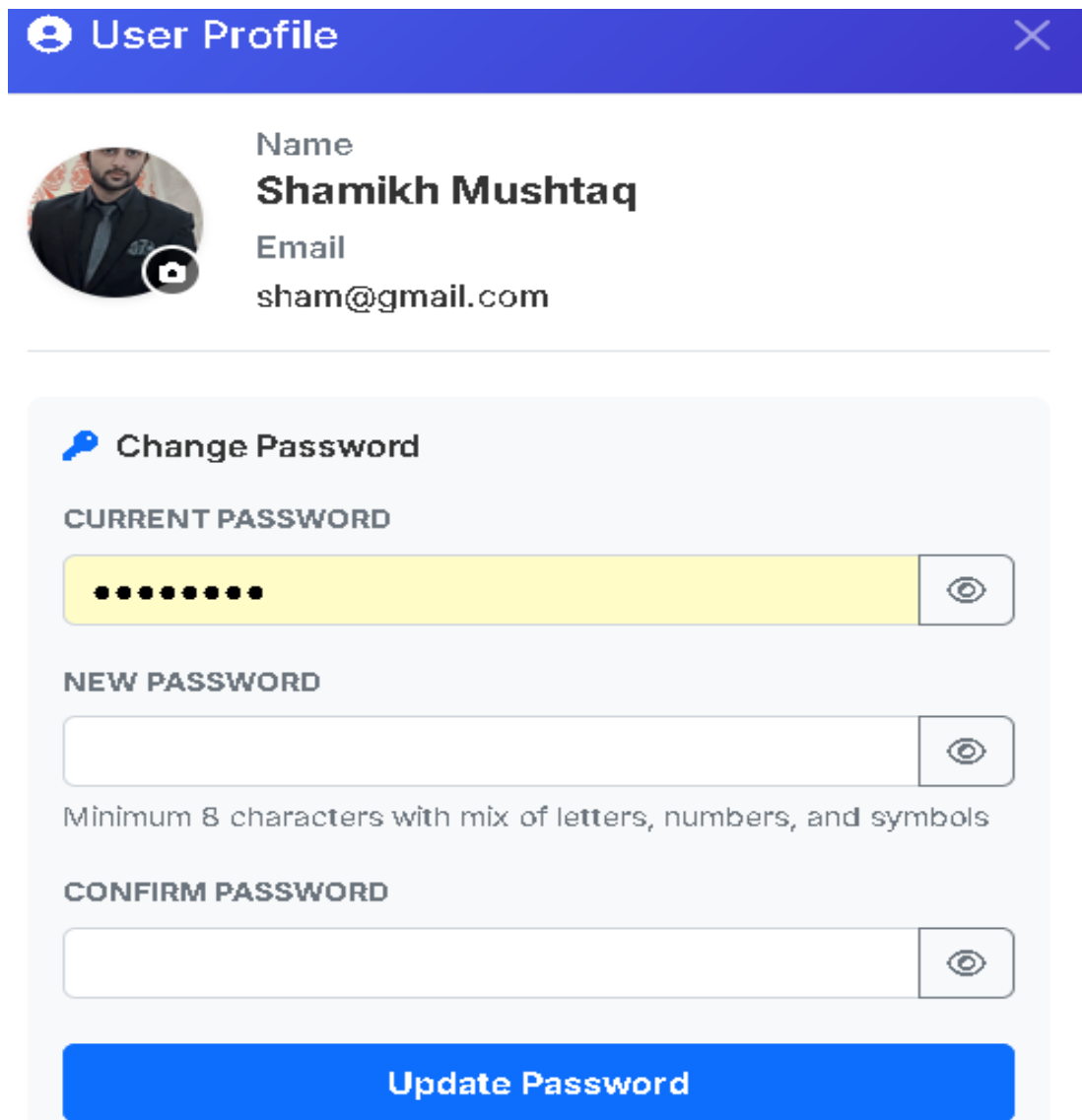


The image shows the next step in the password recovery process. The header and progress indicator are the same as in the previous form, but the second circle in the progress indicator is now dark blue with the number "2", and the third circle is light blue with the number "3". Below the progress indicator, the label "New Password" is followed by a text input field with a masked password (eight dots) and a toggle icon on the right. Below the input field, the text "Medium" is displayed in orange. Underneath, the label "Confirm New Password" is followed by another text input field with a masked password (eight dots) and a toggle icon on the right. At the bottom, there is a large blue button with a white padlock icon and the text "Update Password".

Fig 5.1.2 (c) Password Recovery Form

5.1.3 Update Password Form

- **Description:** Admin can update the account password by entering a new password and confirming it.
- **Fields:**
 - **New Password** (required, min 8 characters)
 - **Confirm Password** (required, must match New Password)
- **Screenshot**



The screenshot displays a web interface for updating a user's password. At the top, a blue header bar contains a user profile icon, the text "User Profile", and a close button (X). Below the header, the user's profile information is shown: a circular profile picture of a man in a suit, the name "Shamikh Mushtaq", and the email "sham@gmail.com". A horizontal line separates the profile section from the "Change Password" form. The form has a light gray background and a blue key icon next to the title "Change Password". It contains three password input fields: "CURRENT PASSWORD" (filled with black dots), "NEW PASSWORD" (empty), and "CONFIRM PASSWORD" (empty). Each field has a toggle icon (an eye) to the right. Below the "NEW PASSWORD" field, a note states: "Minimum 8 characters with mix of letters, numbers, and symbols". At the bottom of the form is a large blue button labeled "Update Password".

Fig 5.1.3 Update Password

5.1.4 Update Profile Picture Form

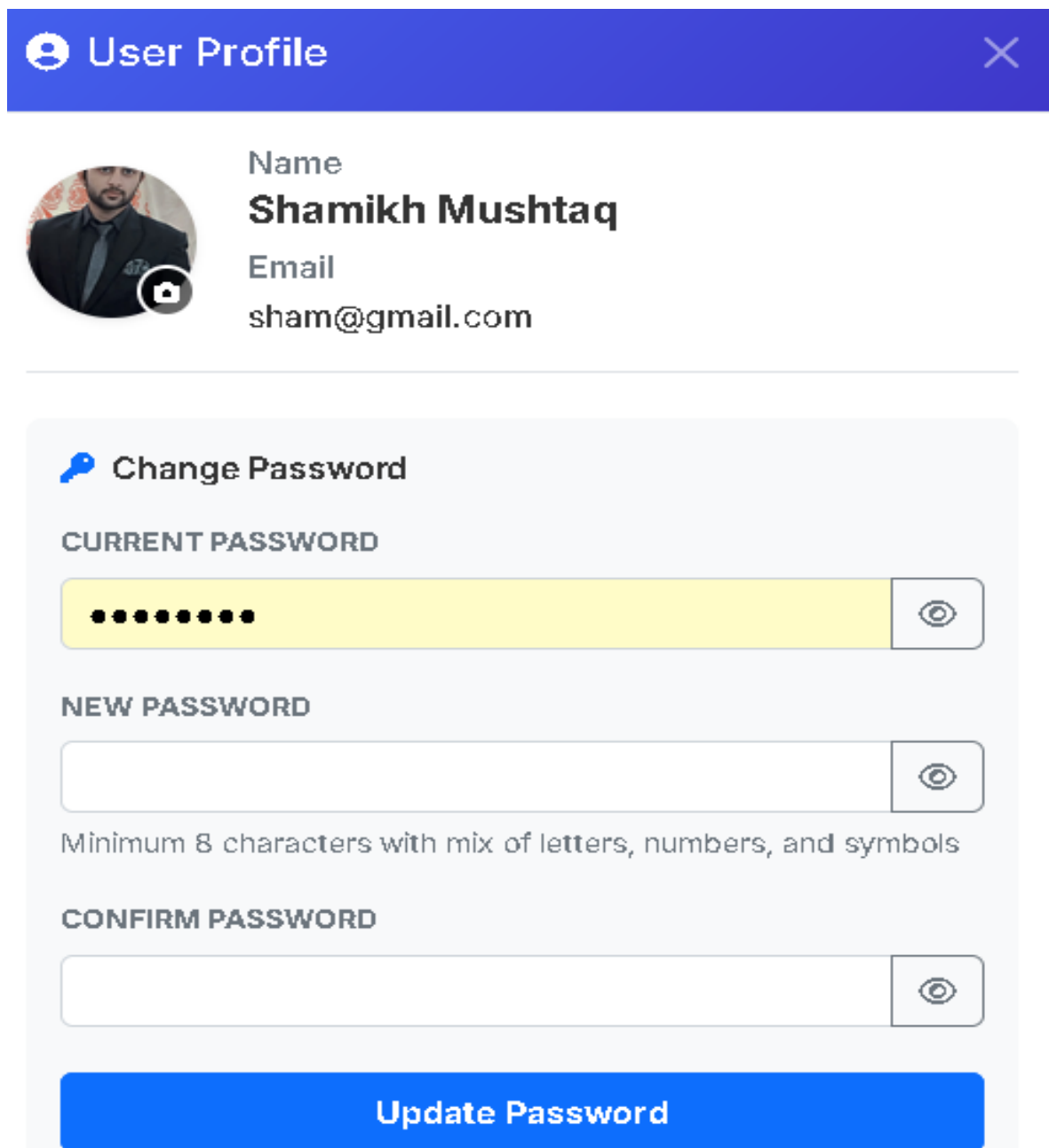
- **Description:**

This feature allows the user to update their profile picture in the SmartNet system for personalization and easier identification.

- **Fields:**

- **Profile Picture** (Image File)

- **Screenshot:**



The screenshot displays a 'User Profile' form with a blue header bar containing a user icon and a close button. Below the header, the user's profile information is shown: a circular profile picture of a man in a suit, the name 'Shamikh Mushtaq', and the email 'sham@gmail.com'. A horizontal line separates the profile information from the 'Change Password' section. This section has a light gray background and contains three password input fields: 'CURRENT PASSWORD' (filled with black dots), 'NEW PASSWORD' (empty), and 'CONFIRM PASSWORD' (empty). Each field has a toggle icon for visibility. Below the 'NEW PASSWORD' field, a note states: 'Minimum 8 characters with mix of letters, numbers, and symbols'. At the bottom of the section is a large blue button labeled 'Update Password'.

Fig 5.1.4 Update Profile Picture Form

5.1.5 System Services Management GUI

- **Description:**

This GUI provides a simple interface to start and monitor the XAMPP server, backend server, and packet capture script. It replaces the need for manual terminal commands, making system management easier and faster.

- **Fields:**

- **Start XAMPP Server** button
- **Start Backend Server** button
- **Start Packet Capture** button
- **Integrated Terminal Windows** for each service to display real-time output logs

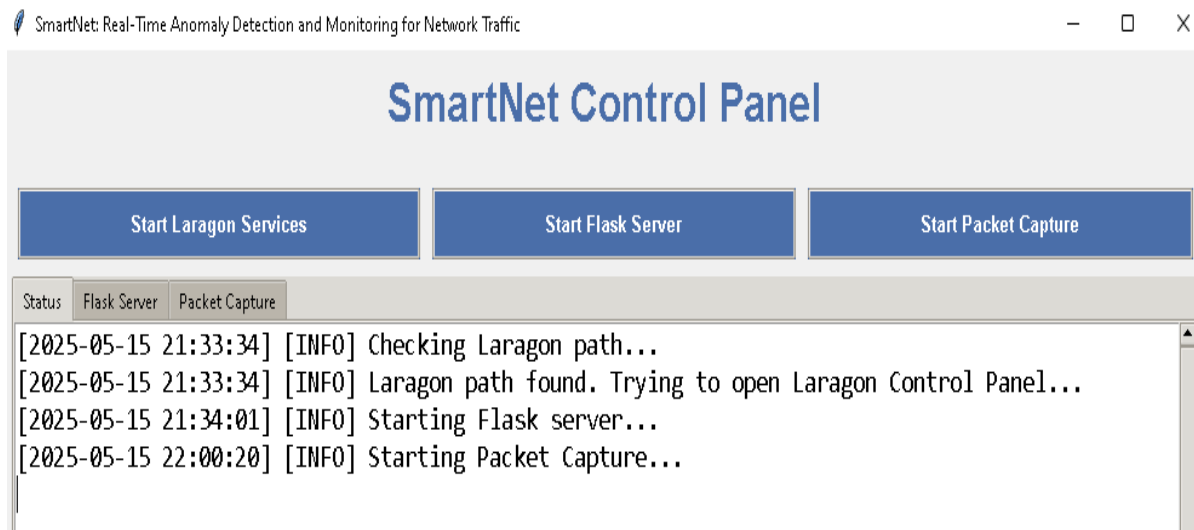


Fig 5.1.5 (a) System Services Management GUI

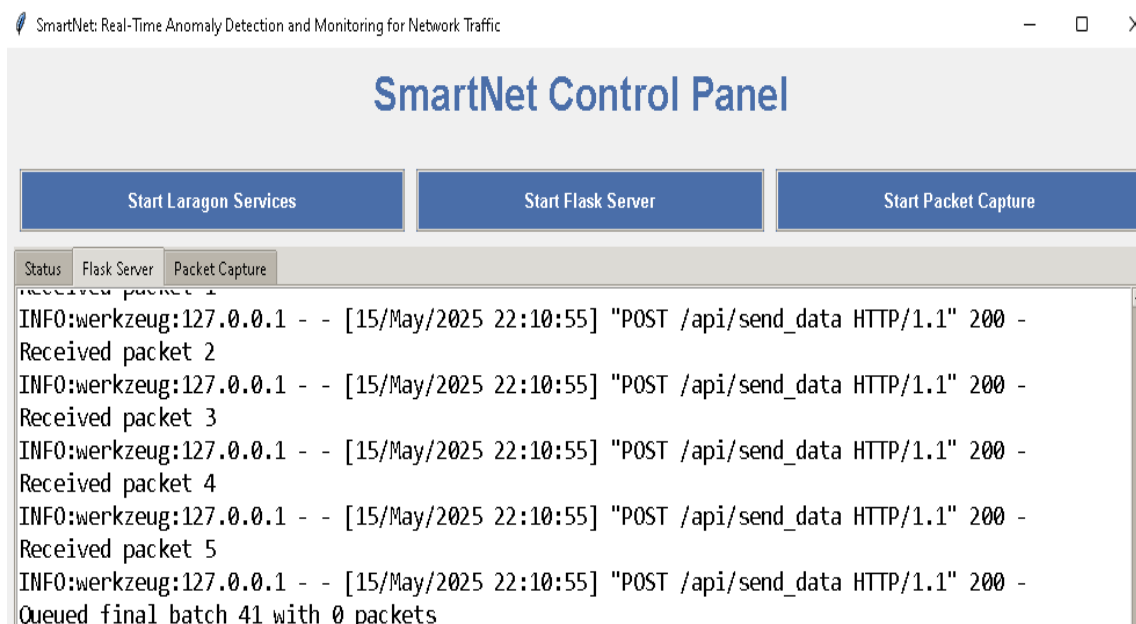


Fig 5.1.5 (b) System Services Management GUI

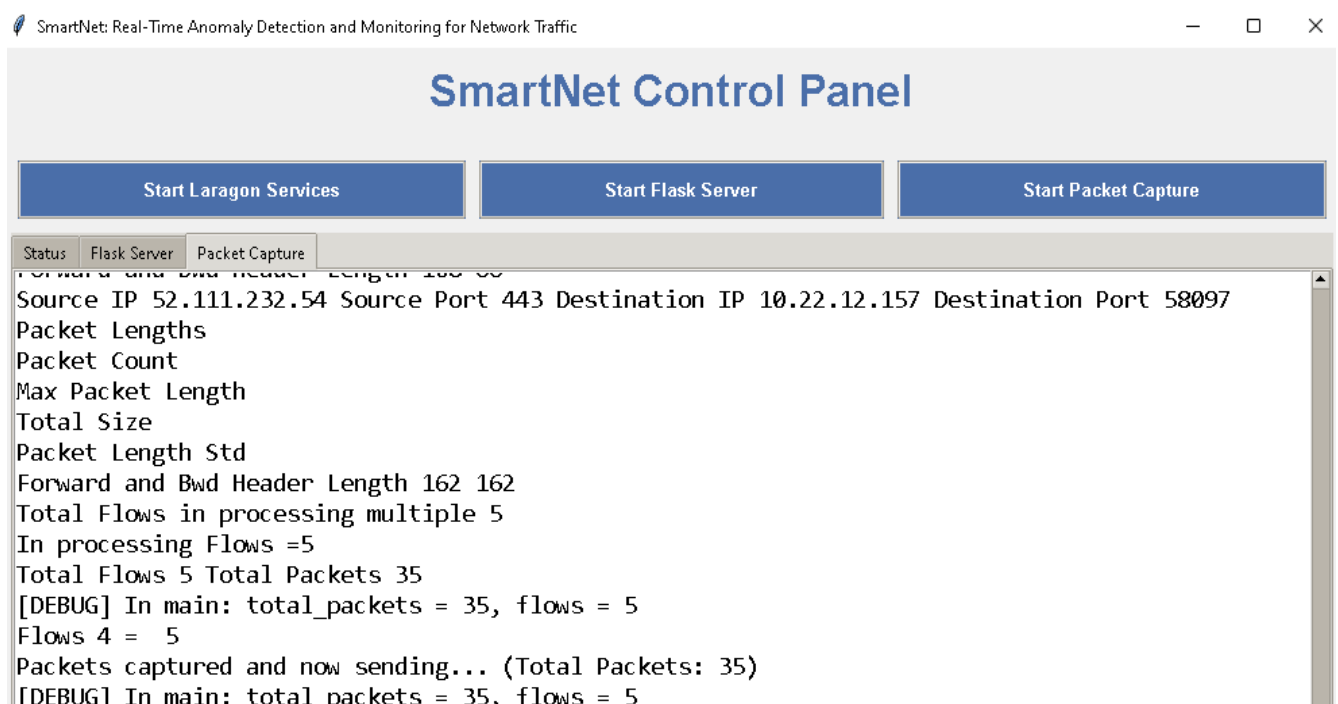


Fig 5.1.5 (c) System Services Management GUI

5.2 Reports

This section provides detailed descriptions and visualizations of the various reports generated by the system. It includes tables and graphs that display key data points, such as network traffic patterns, system performance, recent threats, and machine learning model metrics. Each report is designed to give users a clear understanding of the system's operation and security status, along with the ability to filter, search, and export data for further analysis.

5.2.1 TCP vs UDP Packets Graph

Description:

This pie chart shows the distribution of TCP and UDP packets captured during the current hour. Yellow represents UDP traffic, while blue represents TCP. The total packets and their percentages are displayed below, and hovering over a section shows the exact count of packets for each type.

Screenshot:

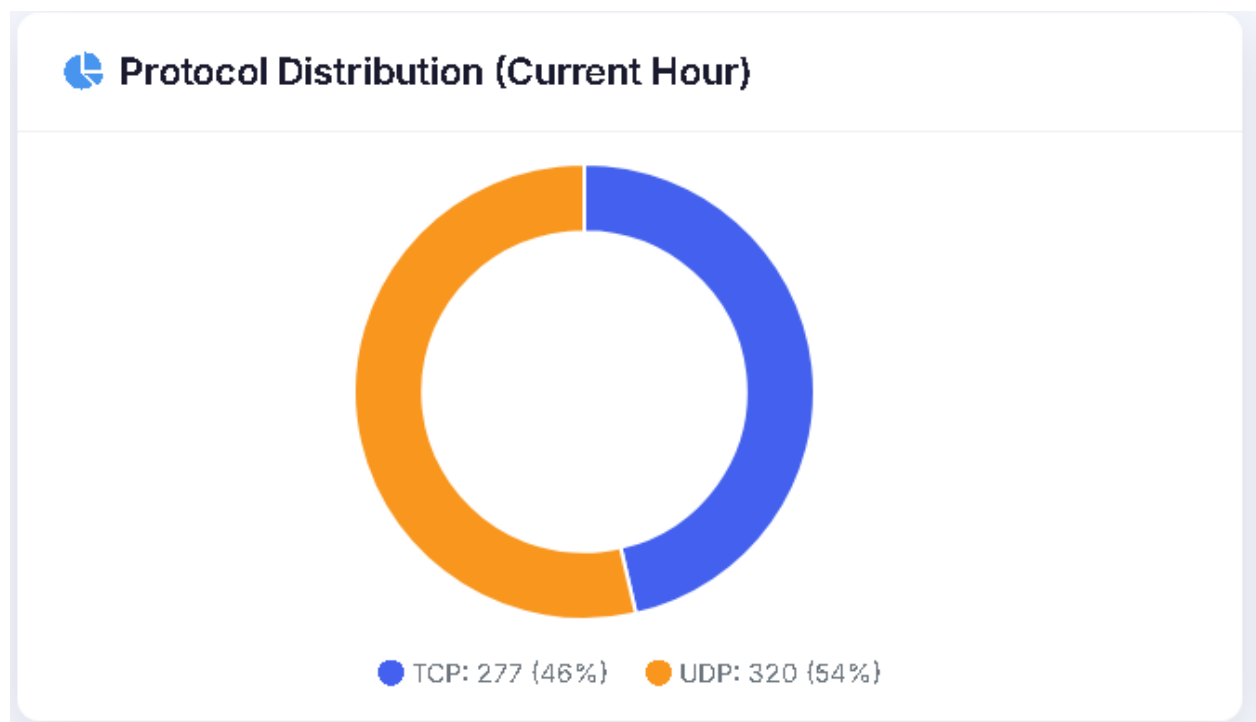


Fig 5.2.1 TCP vs UDP Packets Graph

5.2.2 Packets per Second Line Graph

Description:

This line graph displays the number of packets captured per second over time. It provides a real-time view of network traffic, showing fluctuations in packet transmission. The graph updates continuously to reflect the current rate of packet capture.

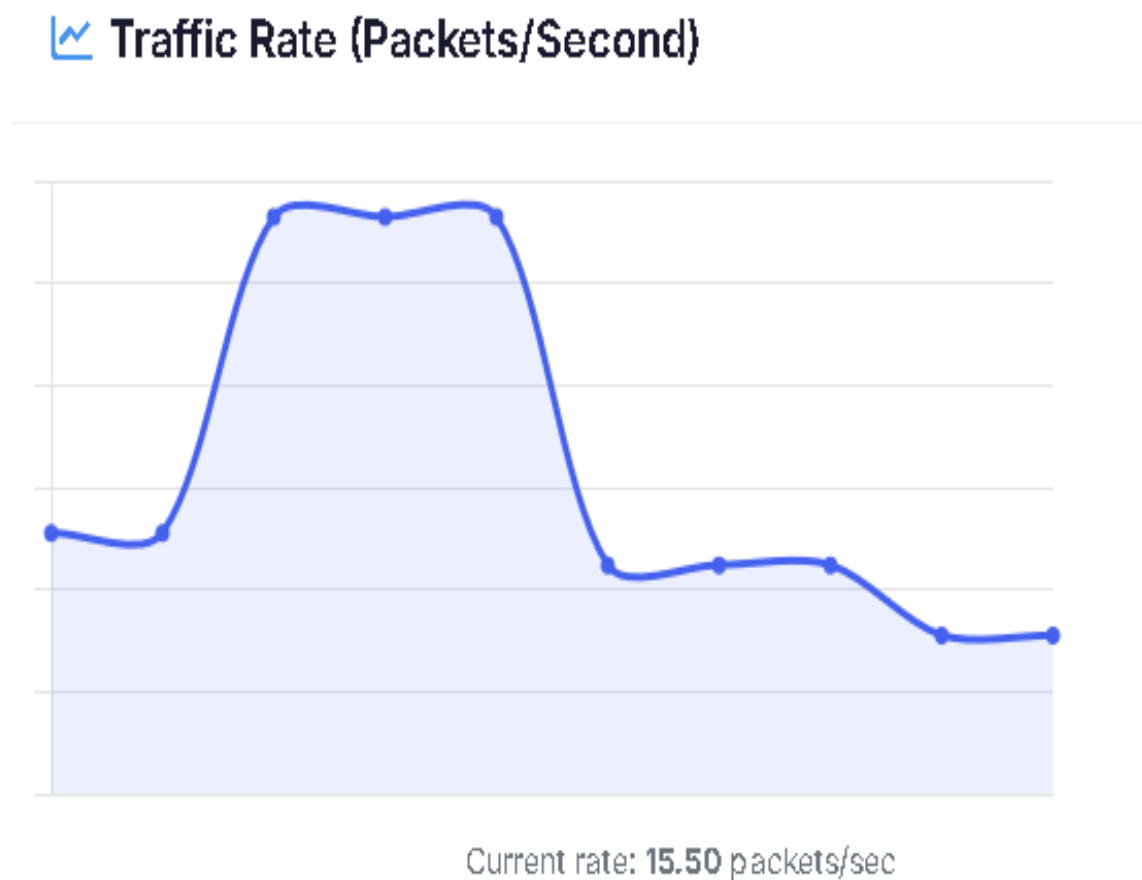
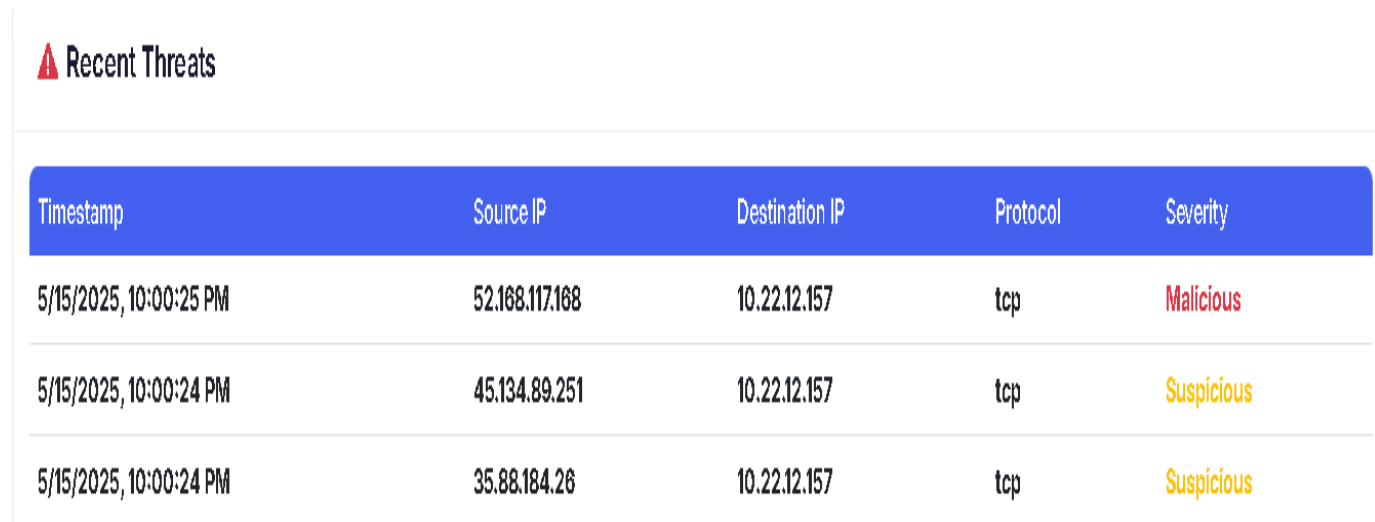
Screenshot:

Fig 5.2.2 Packets per Second Line Graph

5.2.3 Recent Threats Table

Description:

The Recent Threats Table displays a list of detected threats, showing details like the timestamp, source IP, destination IP, protocol, and severity (malicious or suspicious). This table helps users quickly identify potential threats and monitor network security in real-time.

Screenshot:

Recent Threats				
Timestamp	Source IP	Destination IP	Protocol	Severity
5/15/2025, 10:00:25 PM	52.168.117.168	10.22.12.157	tcp	Malicious
5/15/2025, 10:00:24 PM	45.134.89.251	10.22.12.157	tcp	Suspicious
5/15/2025, 10:00:24 PM	35.88.184.26	10.22.12.157	tcp	Suspicious

Fig 5.2.3 Recent Threats Table

5.2.4 ML Model Details Page

Description:

The ML Model Details Page provides insights into the model used for prediction, including the importance of each of the 15 features required for accurate prediction. It also displays key performance metrics such as accuracy, F1 score, correct predictions, true positives, detection rate, balance metric, and discrimination. Additionally, the model type (XGBoost), test size (30%), train size (80%), and the number of estimators (100) are also shown to give a complete overview of the model's configuration and performance.

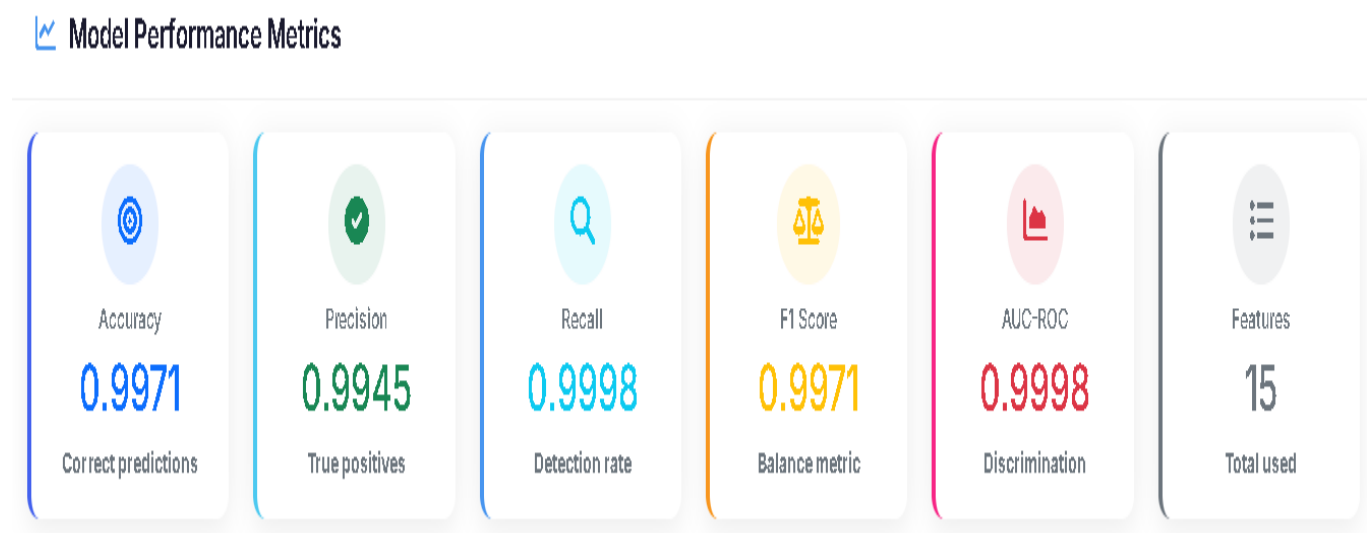
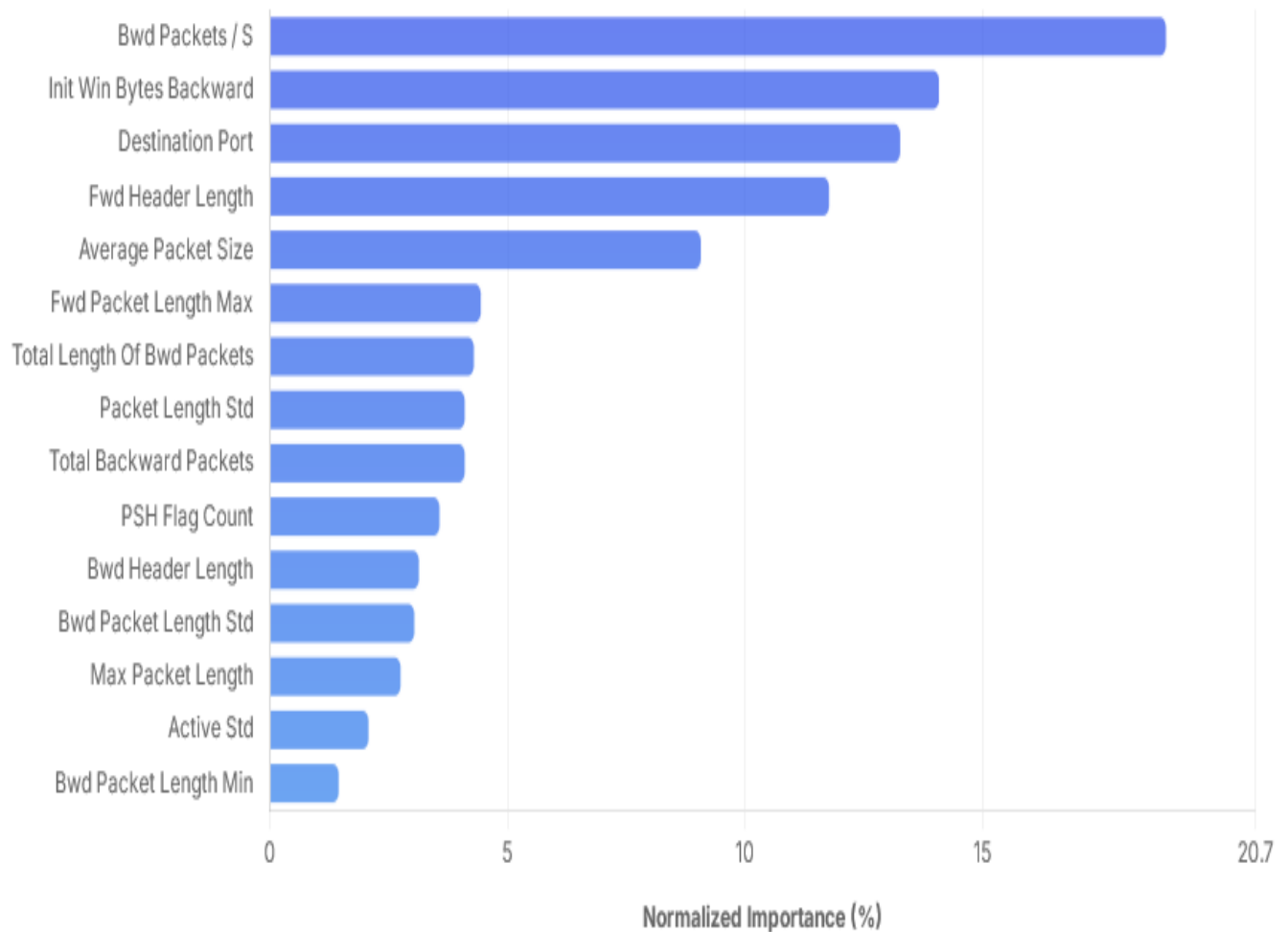
Screenshot:

Fig 5.2.4 (a) ML Model Details Page

★ Feature Importance Analysis



i Higher values indicate more important features for predictions

Fig 5.2.4 (b) ML Model Details Page

Model Specifications

Model Configuration

 Model Type XGBoost

Algorithm Classifier

 Estimators 100

 Test Size 20%

 Last Trained 27-04-2025

Performance Summary

 Accuracy 0.9971

 Precision 0.9945

 F1 Score 0.9971

Fig 5.2.4 (c) ML Model Details Page

Feature Descriptions

Feature Name	Description	Normal Range	Attack Indicator	Importance
Bwd Packet Length Min	Minimum size of packets in backward direction (bytes)	0-100 bytes	Very small packets (<10 bytes)	18.9%
Bwd Packets/s	Rate of packets flowing backward (packets/second)	0-100 packets	High rates (>1000 packets/s)	14.1%
Bwd Packet Length Std	Standard deviation of backward packet sizes	0-50 bytes	Large variations (>100 bytes)	13.3%
Average Packet Size	Mean size of packets in both directions	50-500 bytes	Very small or very large packets	11.8%
Packet Length Std	Standard deviation of all packet lengths	0-100 bytes	Extreme variations in packet sizes	9.1%
Init Win Bytes Backward	Initial TCP window size in backward direction	0-65535 bytes	Unusual window sizes	4.4%
Psh Flag Count	Number of packets with PUSH flag set	0-5 per connection	High PUSH flag counts	4.3%

Fig 5.2.4 (d) ML Model Details Page

Fwd Packet Length Max	Maximum size of forward packets (bytes)	0-1500 bytes	Oversized packets (>1500 bytes)	4.1%
Bwd Header Length	Total header length of backward packets	20-60 bytes	Abnormal header sizes	4.1%
Destination Port	Target service port number	0-49151	Uncommon or suspicious ports	3.6%
Total Backward Packets	Count of packets in backward direction	0-100 per session	High volume of backward packets	3.1%
Active Std	Standard deviation of active time between packets	0-1 second	Irregular timing patterns	3.0%
Fwd Header Length	Total header length of forward packets	20-60 bytes	Abnormal header sizes	2.7%
Total Length of Bwd Packets	Total bytes transferred in backward direction	0-100KB	Large data transfers	2.1%
Max Packet Length	Maximum packet length observed in flow	0-1500 bytes	Oversized packets	1.4%

Fig 5.2.4 (e) ML Model Details Page

5.2.5 System Resources Monitoring Page

Description:

The System Resources Monitoring Page provides a comprehensive view of the system's performance. It displays details like CPU cores, system memory, and network status (active or inactive). Additionally, it shows the resource usage for key components:

- **Backend Server:** CPU usage, memory usage, and uptime.
- **Web Panel:** Active sessions, requests per minute, and average response time.
- **Packet Capture:** Packets per second, total packets captured, and threats detected.

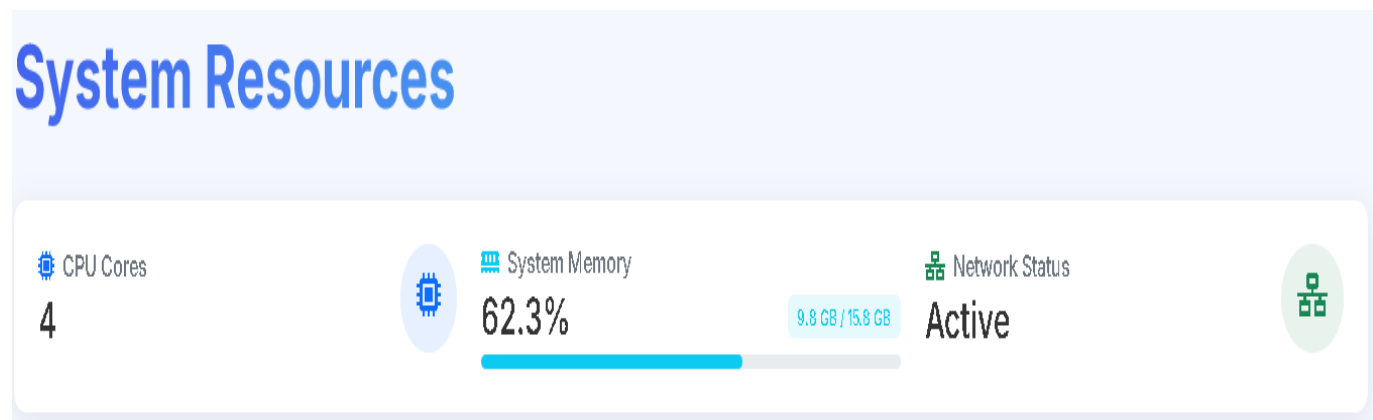
Screenshot:

Fig 5.2.5 (a) System Resources Monitoring Page

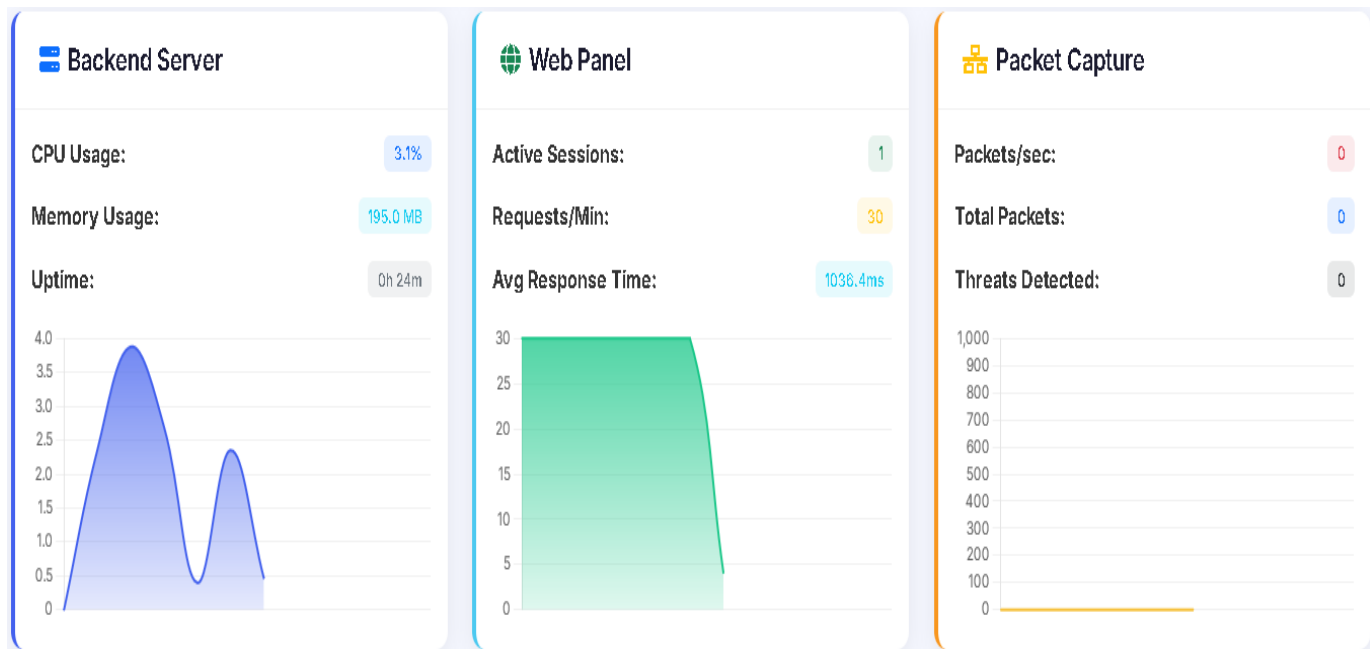


Fig 5.2.5 (b) System Resources Monitoring Page

5.2.6 Complete Flow Data Analysis

Description:

The Complete Flow Data Analysis displays detailed information for each captured flow, including attributes such as source port, destination IP, protocol, timestamp, packet length, and various other packet metrics. The table can be filtered by protocol (TCP, UDP, or all) and severity level (normal, suspicious, or malicious). Users can search specific columns by selecting the column and typing in the search bar (press ESC to unselect). A button is provided to download all data in CSV format. For viewing complete flow details, users can select a row, and a modal will open with a "View JSON Packet" button to display the full flow content.

Screenshot:



Fig 5.2.6 (a) Complete Flow Data Analysis

ID	SOURCE IP	SOURCE PORT	DESTINATION IP	DESTINATION PORT	PROTOCOL	TIMESTAMP	DIRECTION	BWD PACKET MIN	BWD PACKETS/S
45360	148.113.20.116	443	10.22.12.157	56530	TCP	2025-05-15 22:07:43.801	Bidirectional	60	10.2003416
45359	52.182.143.209	443	10.22.12.157	57794	TCP	2025-05-15 22:07:42.673	Bidirectional	60	3.4910185
45358	13.67.9.5	443	10.22.12.157	57897	TCP	2025-05-15 22:07:39.786	Bidirectional	66	2.4985101

Fig 5.2.6 (b) Complete Flow Data Analysis

5.2.7 IP Restrictions Panel

Description:

The IP Restrictions Panel allows administrators to manually block and unblock IP addresses. The interface includes a form to specify the IP, duration (e.g., 24 hours), and reason for blocking. A table displays all active blocks with columns for IP address, timestamps, duration, and an "Unblock" action button. Blocked IPs are enforced via Windows firewall rules (netsh advfirewall) and logged in the database. The panel provides full audit capabilities, tracking which admin performed each action and when.

Screenshot:

Manual IP Blocking

IP Address

Block Duration

Reason

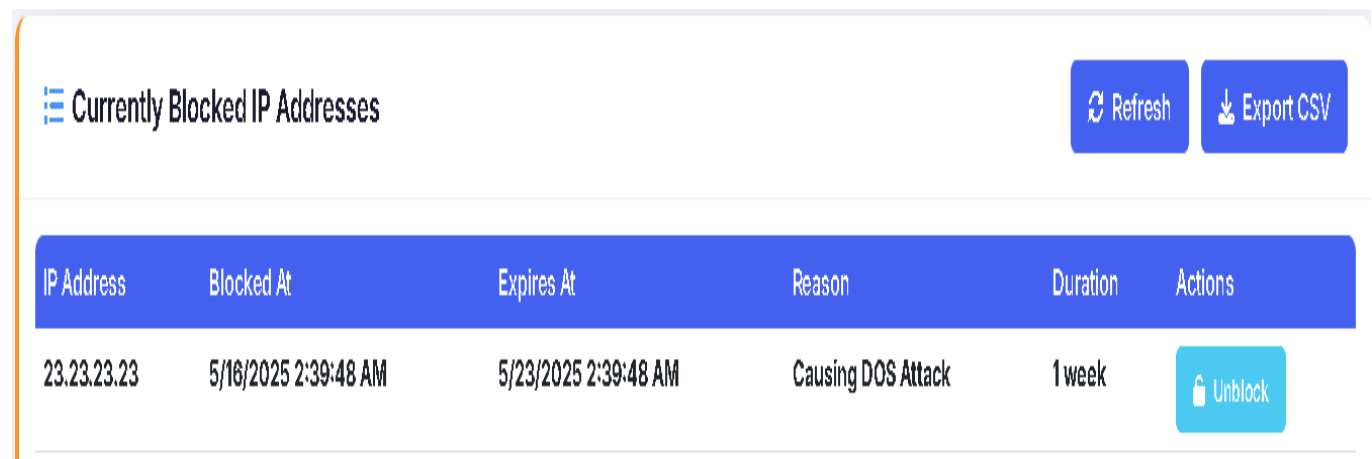
e.g., 192.168.1.100

24 hours

Brief reason for blocking

Block IP Address

Fig 5.2.7 (a) IP Restrictions Panel



The screenshot shows a web interface for managing IP restrictions. At the top left, there is a header 'Currently Blocked IP Addresses' with a hamburger menu icon. To the right of this header are two blue buttons: 'Refresh' with a circular arrow icon and 'Export CSV' with a download icon. Below the header is a table with a blue header row and one data row. The table has six columns: 'IP Address', 'Blocked At', 'Expires At', 'Reason', 'Duration', and 'Actions'. The data row shows the IP address '23.23.23.23', blocked at '5/16/2025 2:39:48 AM', expires at '5/23/2025 2:39:48 AM', reason 'Causing DOS Attack', duration '1 week', and an 'Unblock' button with a lock icon.

IP Address	Blocked At	Expires At	Reason	Duration	Actions
23.23.23.23	5/16/2025 2:39:48 AM	5/23/2025 2:39:48 AM	Causing DOS Attack	1 week	

Fig 5.2.7 (b) IP Restrictions Panel

Chapter 6

Conclusion

6.1 Summary of the Project

This thesis presented the design, development, and implementation of SmartNet, a real-time network traffic analysis system that uses machine learning to classify live network data as normal, suspicious, or malicious. The project addresses the increasing need for intelligent, automated threat detection in networks by integrating packet capture mechanisms, flow-based feature extraction, and a trained XGBoost model into a scalable client-server architecture with an interactive web interface.

Key achievements include:

- **Technical Feasibility:** Demonstrated through the successful training and deployment of an XGBoost classifier using CIC-IDS 2017 dataset for accurate network traffic classification.
- **Operational Usability:** A responsive admin panel enables real-time monitoring, alerting, and detailed flow analysis through an intuitive dashboard.
- **Financial Viability:** Built entirely with open-source tools (Python, MySQL, Flask) ensuring cost-effectiveness and accessibility for future deployment in budget-constrained environments.

The project architecture, outlined in detail across Chapters 3 to 5, emphasizes modular design, real-time data processing, and system responsiveness to ensure seamless traffic analysis and administrator oversight.

6.2 Key Contributions

1. Real-Time Flow-Based Traffic Classification:

- Flow generation using 5-tuple identifiers with time-based segmentation.
- Live feature extraction and batch prediction using a trained ML model.

2. Efficient Threading & Parallelism:

- Asynchronous packet capture and server communication using threading on the client side.
- Concurrent prediction and database writing on the server side for optimal throughput.

3. Interactive Admin Panel:

- Web-based interface for real-time visualization of traffic statistics.
- Features include threat rate monitoring, live packet/second graphs, IP analytics, and alert notifications.
- Modal-based packet deep-dives and JSON/CSV export features.

4. Resource-Aware Design:

- Monitors and visualizes CPU and memory usage of all major system components.
- Balances model complexity with runtime efficiency for deployment on modest hardware.

6.3 Limitations

While the system achieved its functional goals, a few limitations exist:

- **Manual IP Management:** Blocking/unblocking requires admin intervention; no automatic threat-response mechanism exists.
- **Limited Protocol Scope:** Current implementation focuses only on TCP and UDP packets, leaving out ICMP, ARP, etc.
- **Static Model Thresholds:** Classification thresholds are hardcoded; adaptive thresholding was not explored.
- **Training Dataset Dependence:** Performance may vary with traffic types absent in the CIC-IDS 2017 dataset.
- **No Deep Packet Inspection (DPI):** Payloads are not analyzed, allowing content-level attacks to bypass detection.

6.4 Future Work

To overcome limitations and expand capabilities, future directions include:

1. **Automated Threat Response:**

- **Auto-blocking:** Integrate with IDS/IPS to block malicious IPs automatically based on severity thresholds.
- **Temporary Blocks:** Implement time-bound auto-unblocking after threat mitigation.

2. **Protocol Expansion:** Support for ICMP, ARP, and encrypted traffic patterns.

3. **Adaptive Learning:**

- Incremental learning for model adaptability.
- Risk-based threshold tuning.

4. **Payload Analysis:**

- Deep packet inspection for content-level threats (e.g., phishing).

5. **Client Ecosystem:**

- Lightweight desktop/mobile apps for distributed monitoring.

6. **Security Enhancements:**

- Encrypted client-server communication.
- Role-based access control for admins.

6.5 *Final Remarks*

In an era of growing cybersecurity threats, the ability to analyze and respond to network intrusions in real-time is crucial. This project introduces SmartNet, a system that brings machine learning-based intelligence to network traffic monitoring in a practical, extensible, and cost-effective way.

By combining real-time packet capture, feature engineering, intelligent classification, and a user-friendly dashboard, SmartNet empowers organizations and administrators with actionable network insights. Though improvements are possible, this work lays the foundation for future enhancements in real-time intrusion detection systems. With further research and development, SmartNet can evolve into a powerful tool for proactive network defense and anomaly detection in modern cyber environments

References

Wireshark Foundation. (n.d.). TShark - Terminal-based packet capture. Retrieved from <https://www.wireshark.org/docs/man-pages/tshark.html>

Flask Documentation. (n.d.). Flask web framework. Retrieved from <https://flask.palletsprojects.com/en/stable/>

Scikit-learn Developers. (n.d.). scikit-learn: Machine learning in Python. Retrieved from <https://scikit-learn.org/stable/>

Bootstrap Team. (n.d.). Bootstrap – Powerful front-end framework. Retrieved from <https://getbootstrap.com/>

Python Software Foundation. (n.d.). bcrypt – Password hashing for security. Retrieved from <https://pypi.org/project/bcrypt/>

Python Software Foundation. (n.d.). tkinter – Python interface to Tcl/Tk. Retrieved from <https://docs.python.org/3/library/tkinter.html>

Chethu N. (2021). Network Intrusion Dataset (CIC-IDS 2017) <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset>